

Deep Learning

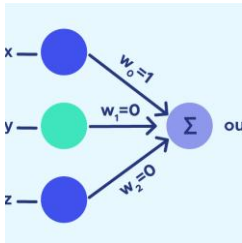


Lecture 02: Neural Networks

Overview

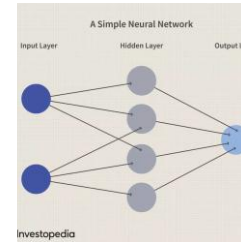


Today:



Neural Networks

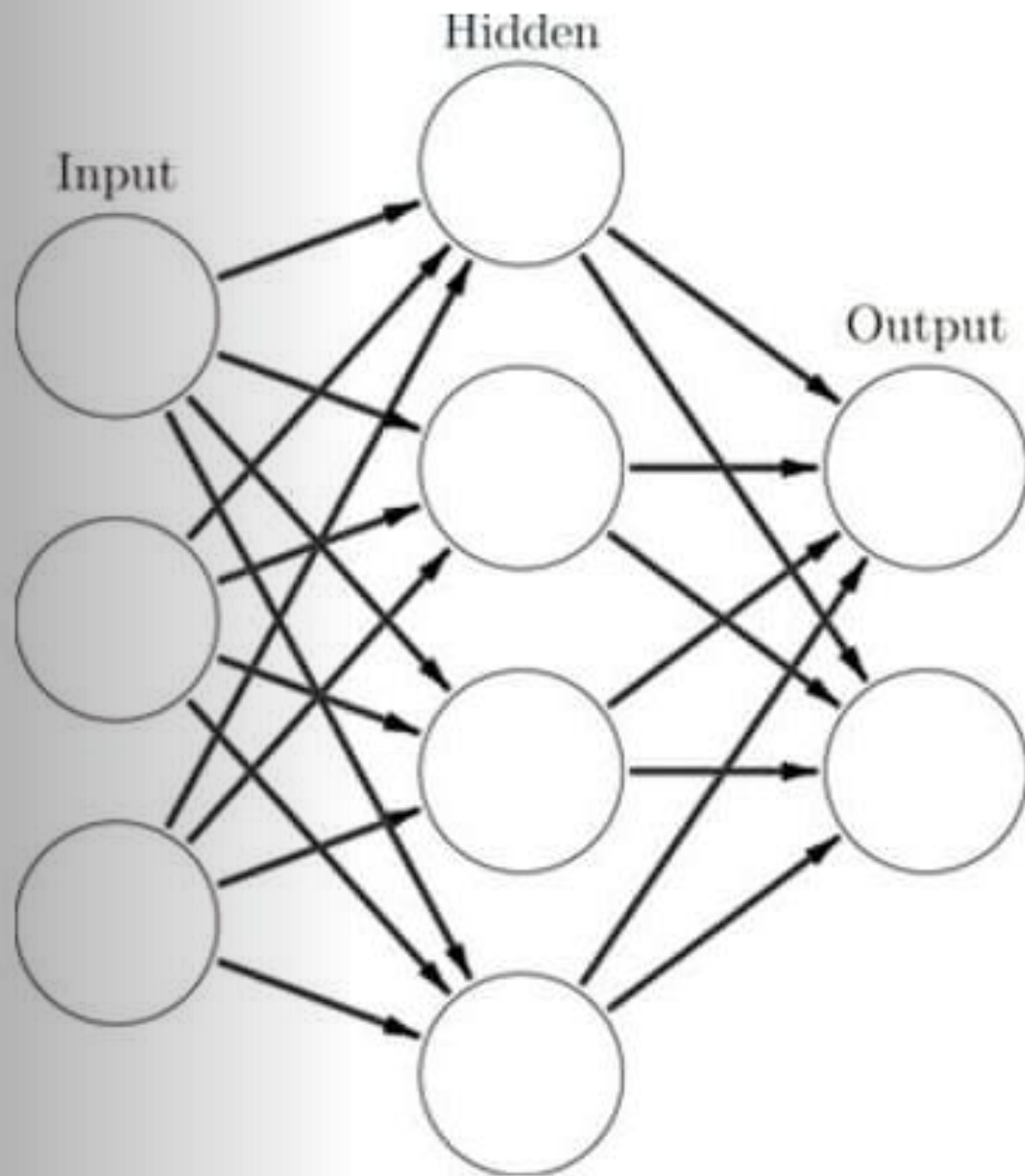
Basics of Neural Network
Perceptron
Mathematics of Perceptron
Gradient Descent



Next Week:

MLPs and ANNs

What are Neural Networks?



Serena



Mike



Jyoti



Shakib



Mohan



Sergey



Nidhi



Chen





Is this Koala?

Yes

No

Is this Koala?



Yes

No





Mike



Mohan



Jyoti

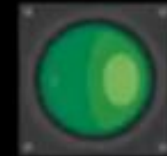


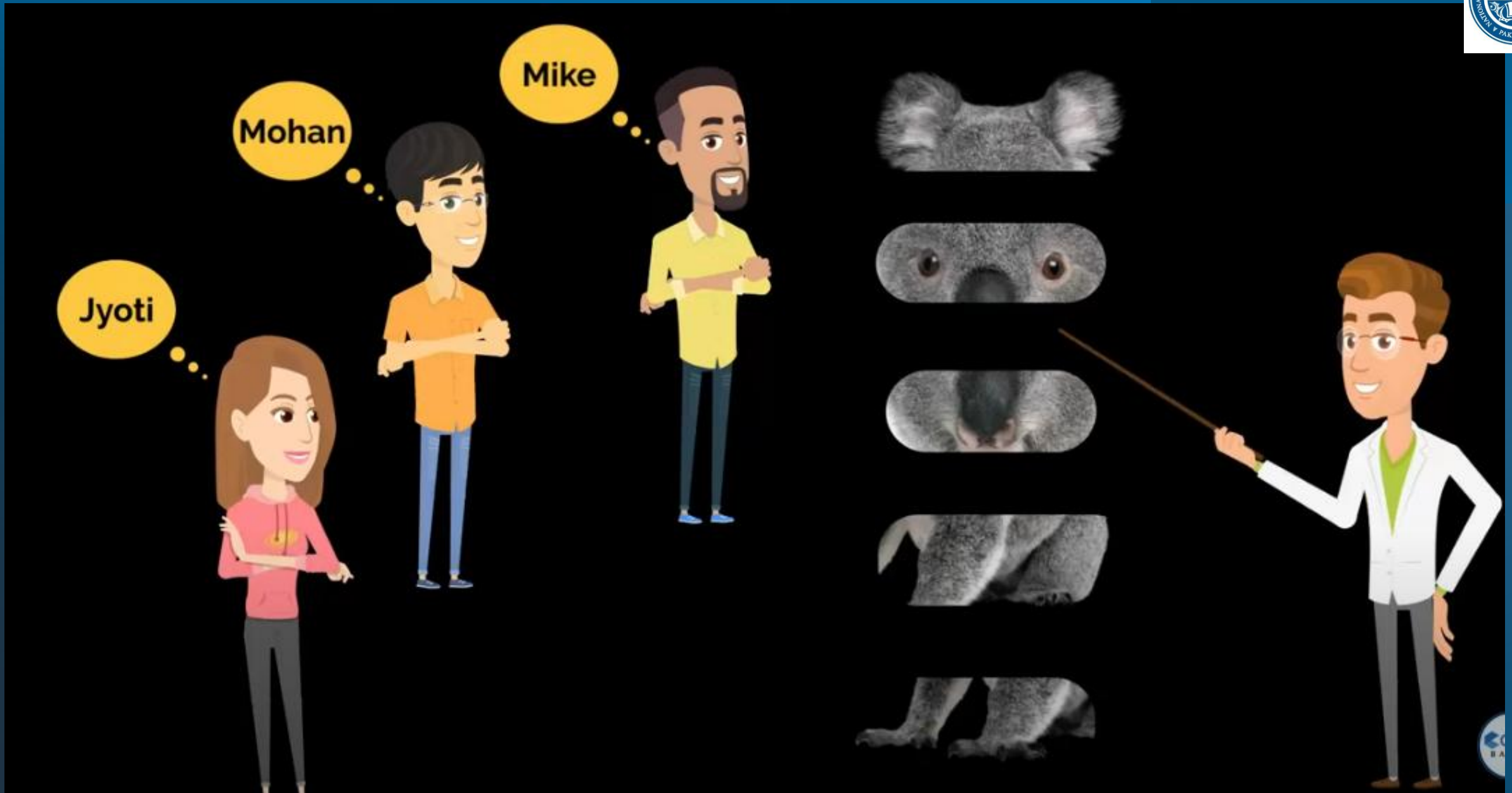
Chen





0 $\dashrightarrow$ 0.5 $\dashrightarrow$ 1





Mike



Mohan



Not a Koala Eyes



0.03

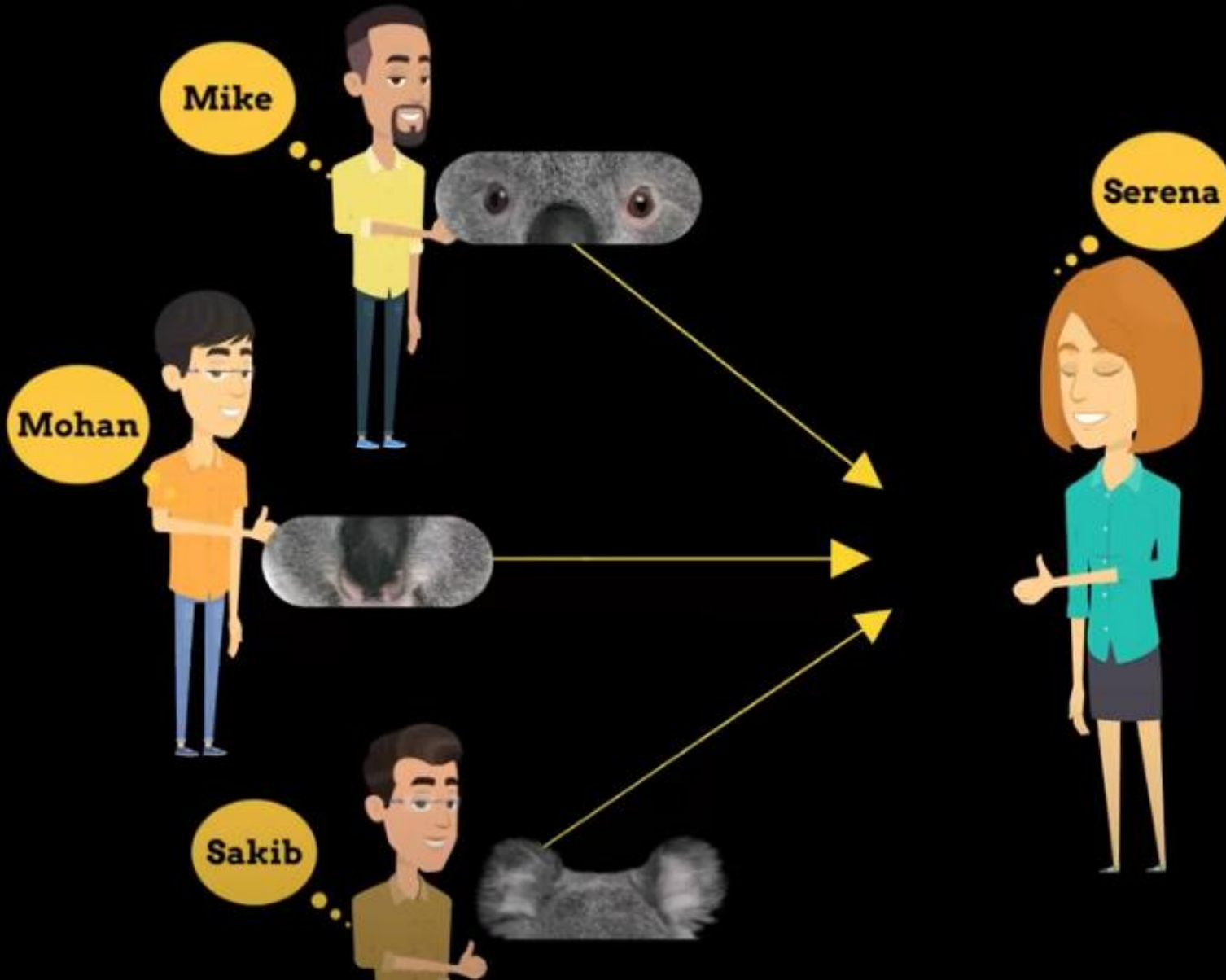


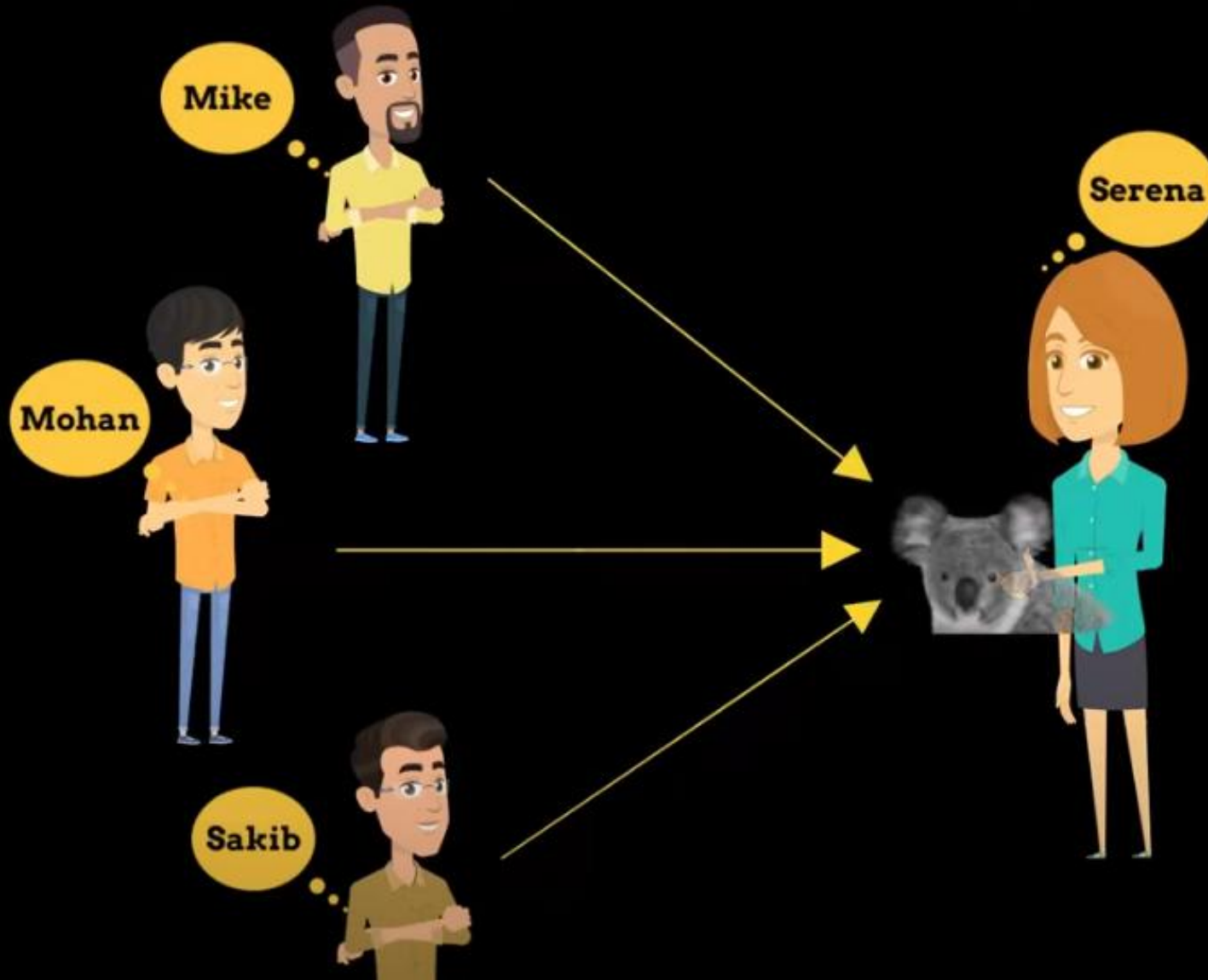
Not a Koala Nose

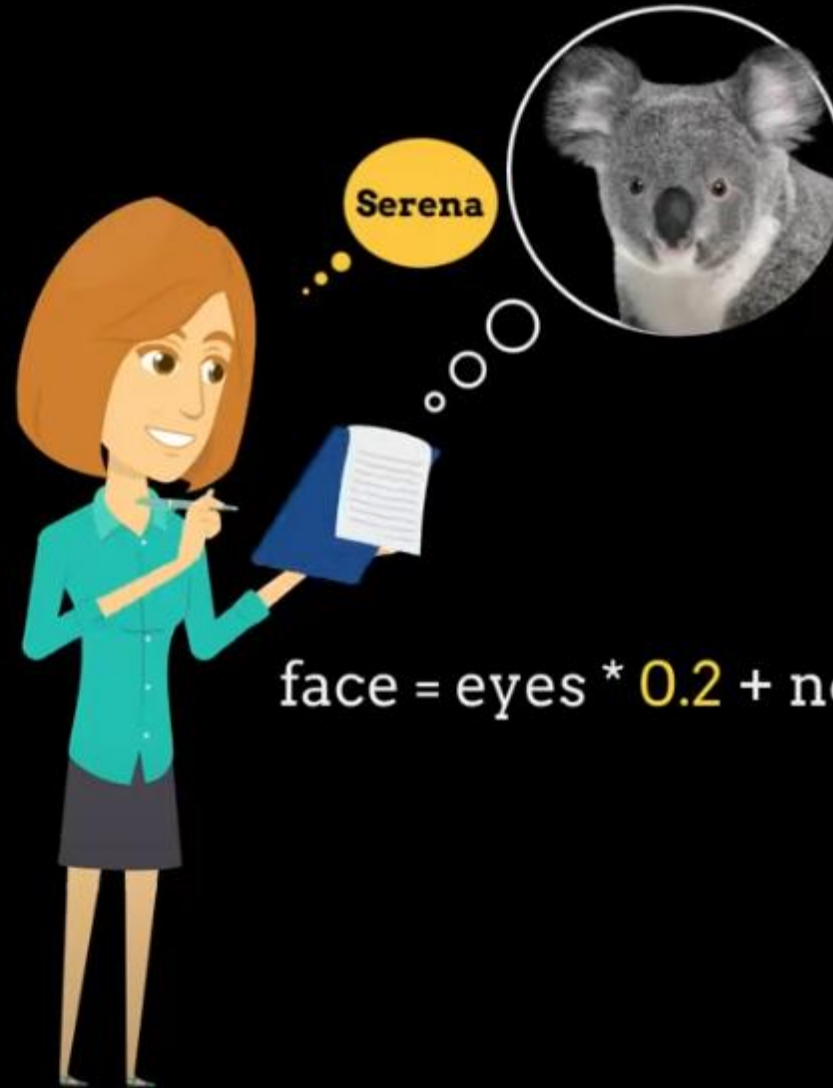


0.1

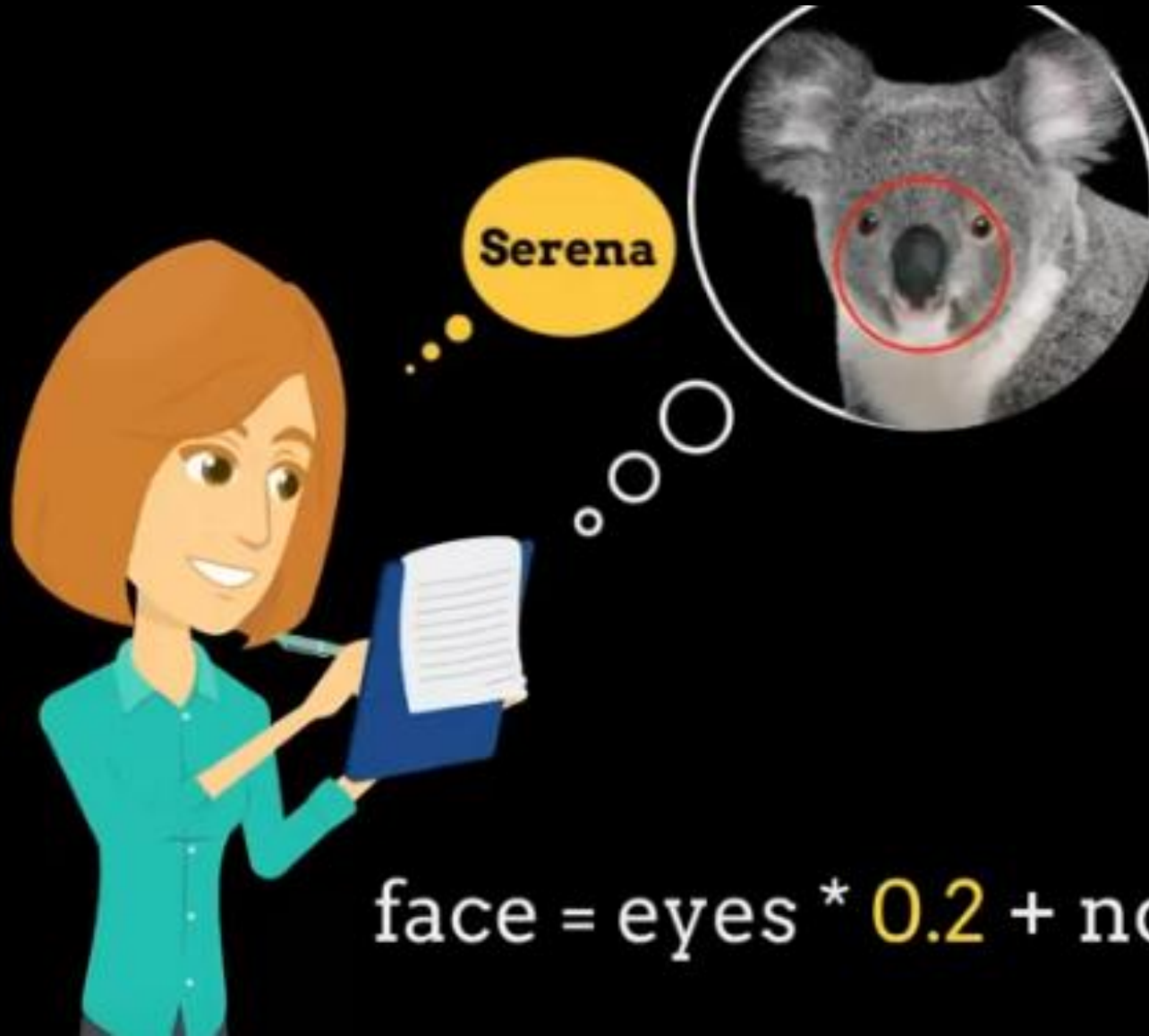




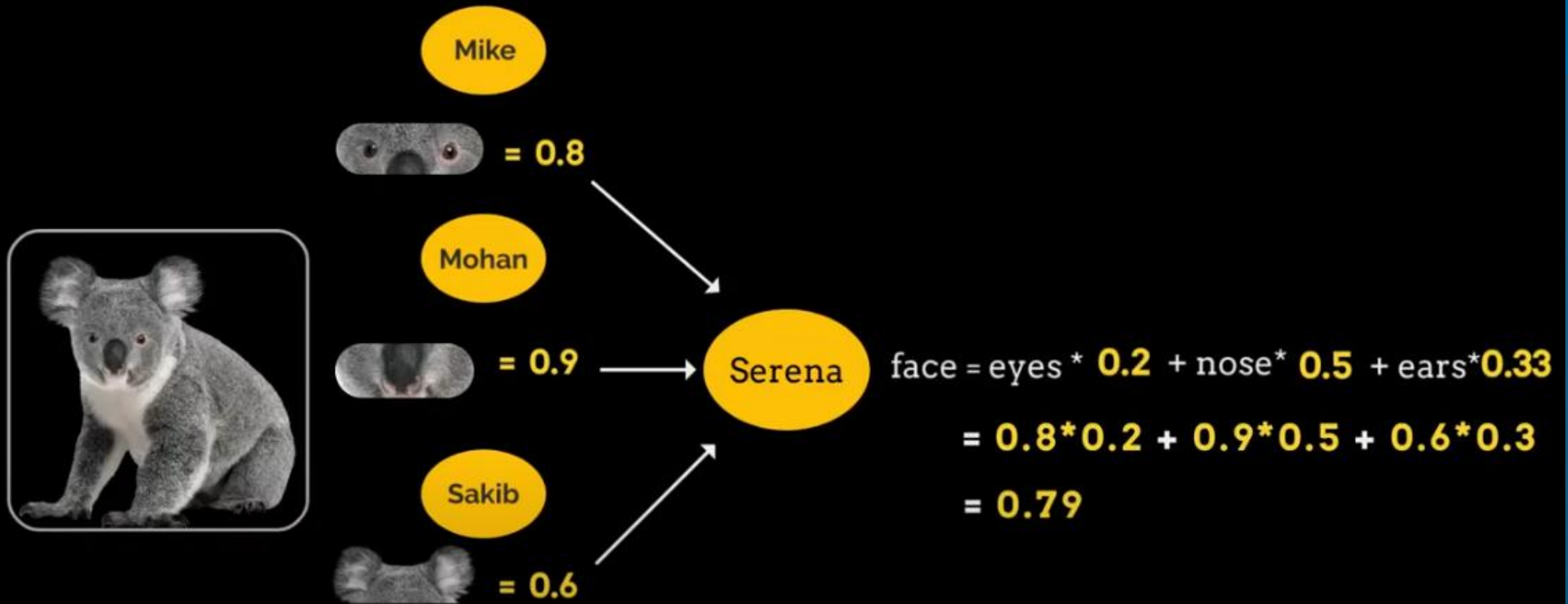


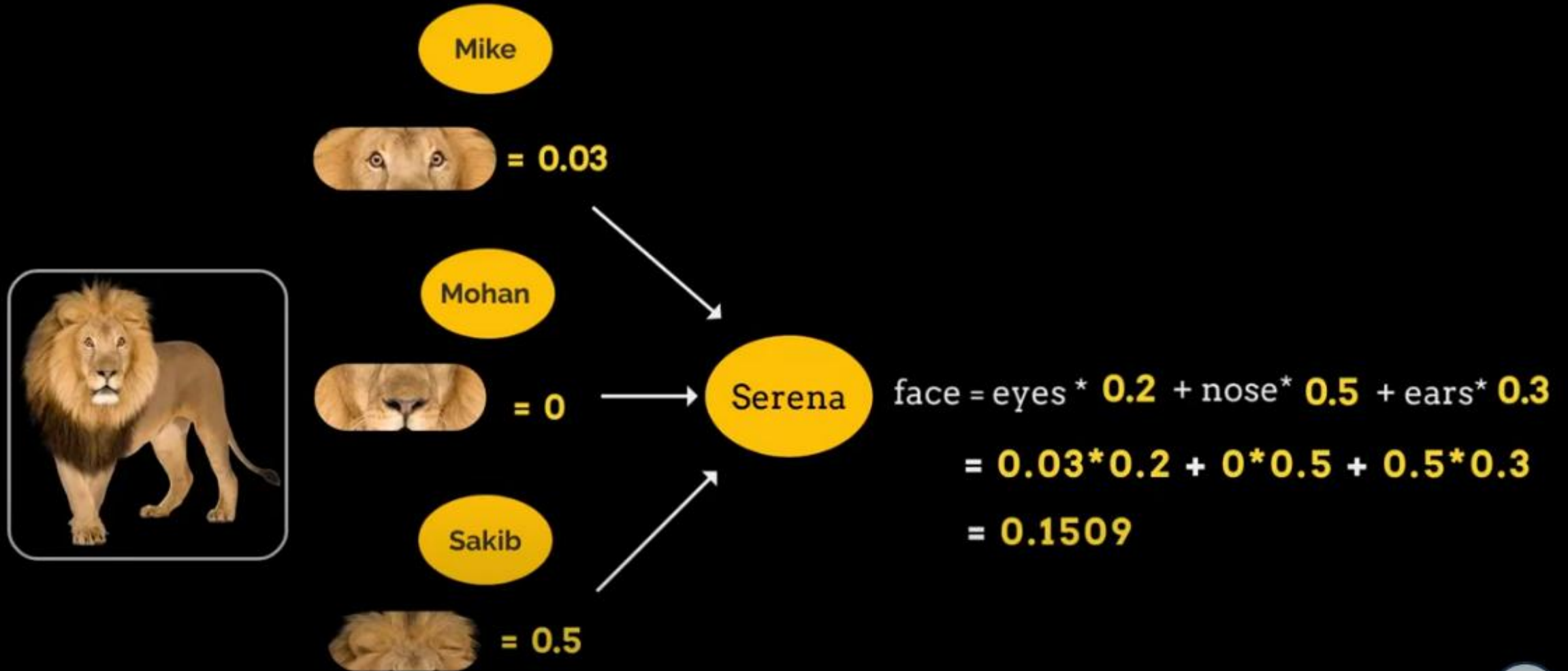


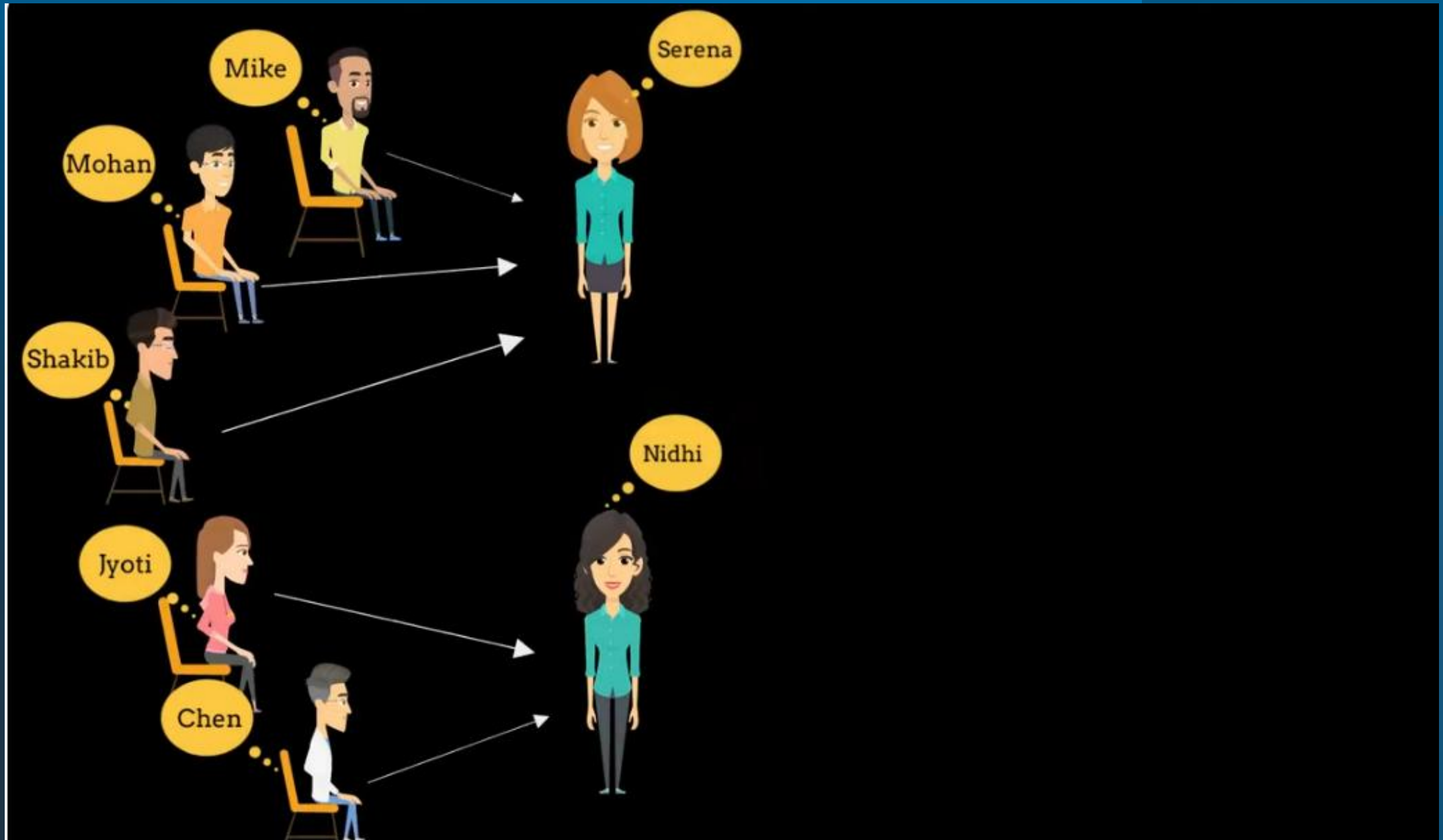
$$\text{face} = \text{eyes} * 0.2 + \text{nose} * 0.5 + \text{ears} * 0.3$$

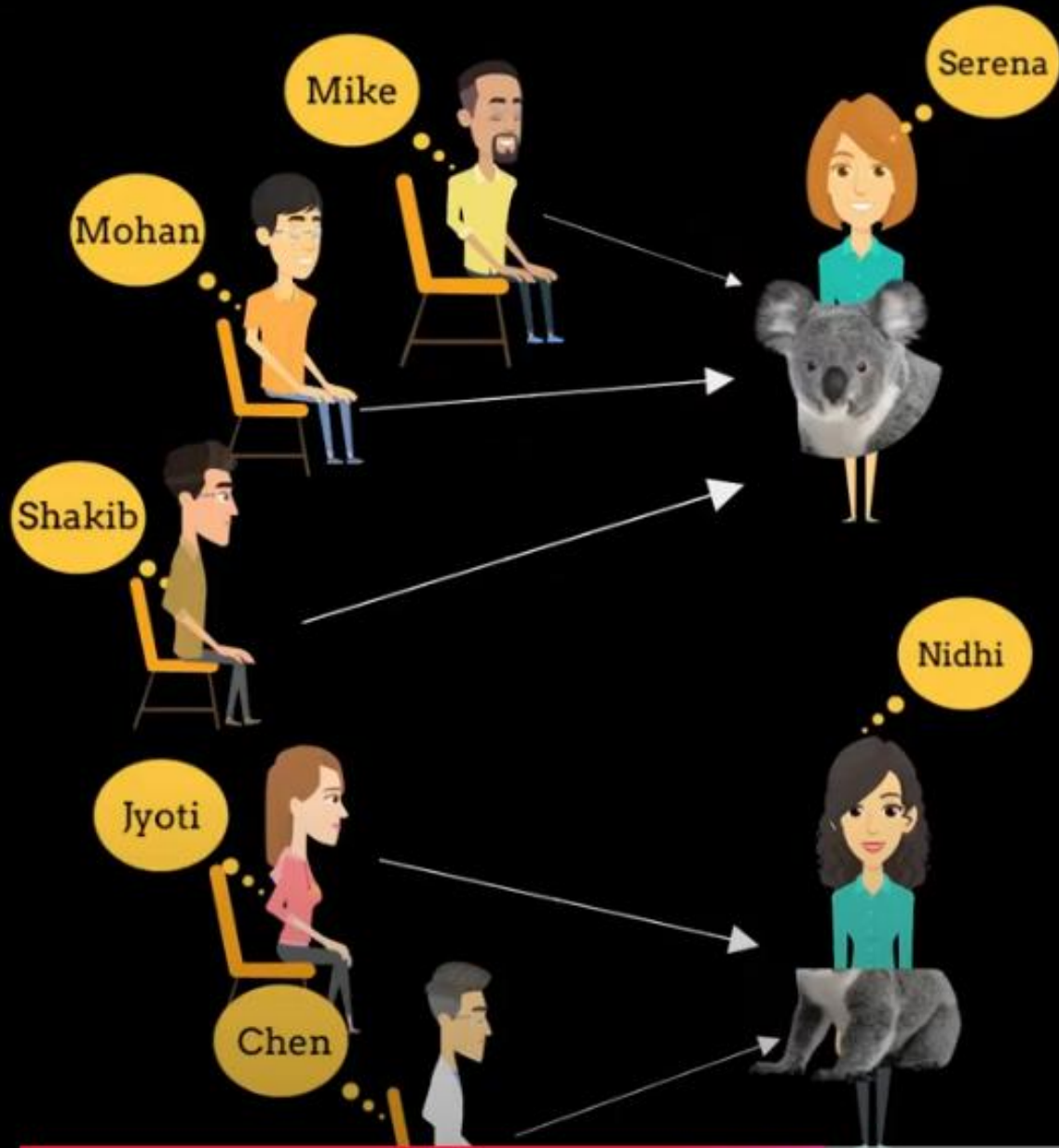


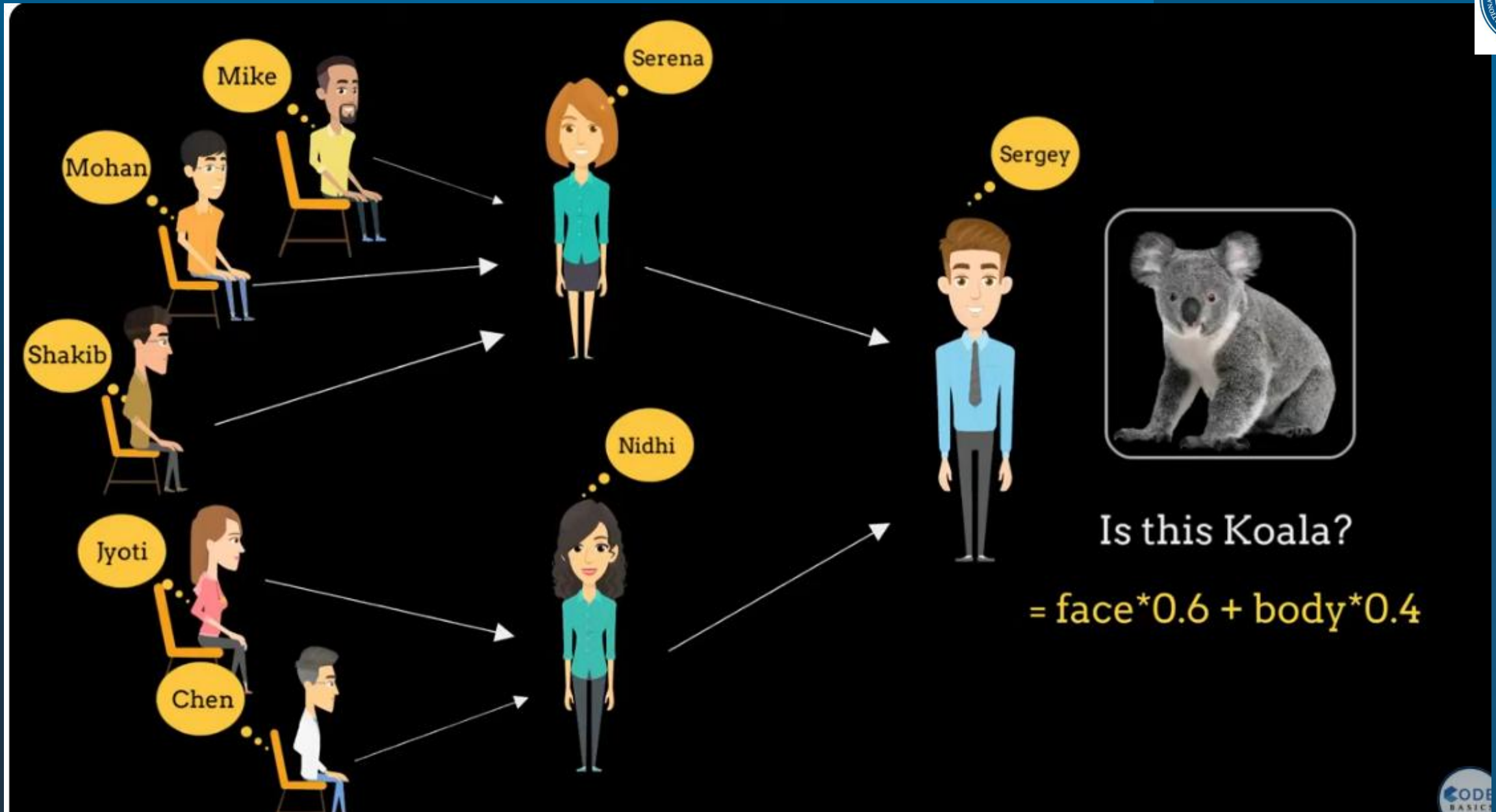
$$\text{face} = \text{eyes} * 0.2 + \text{nose} * 0.5 + \text{ears} * 0.3$$







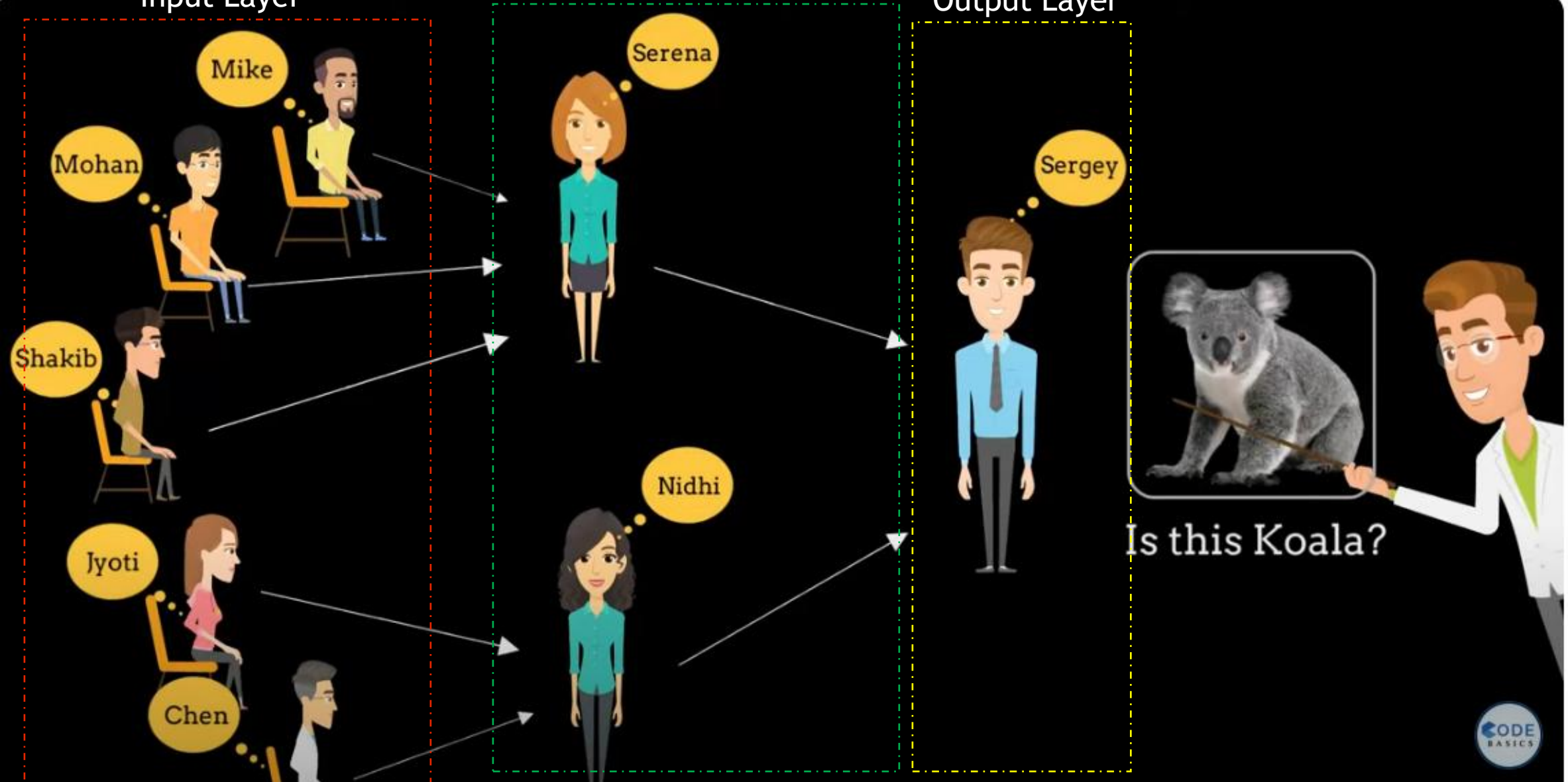




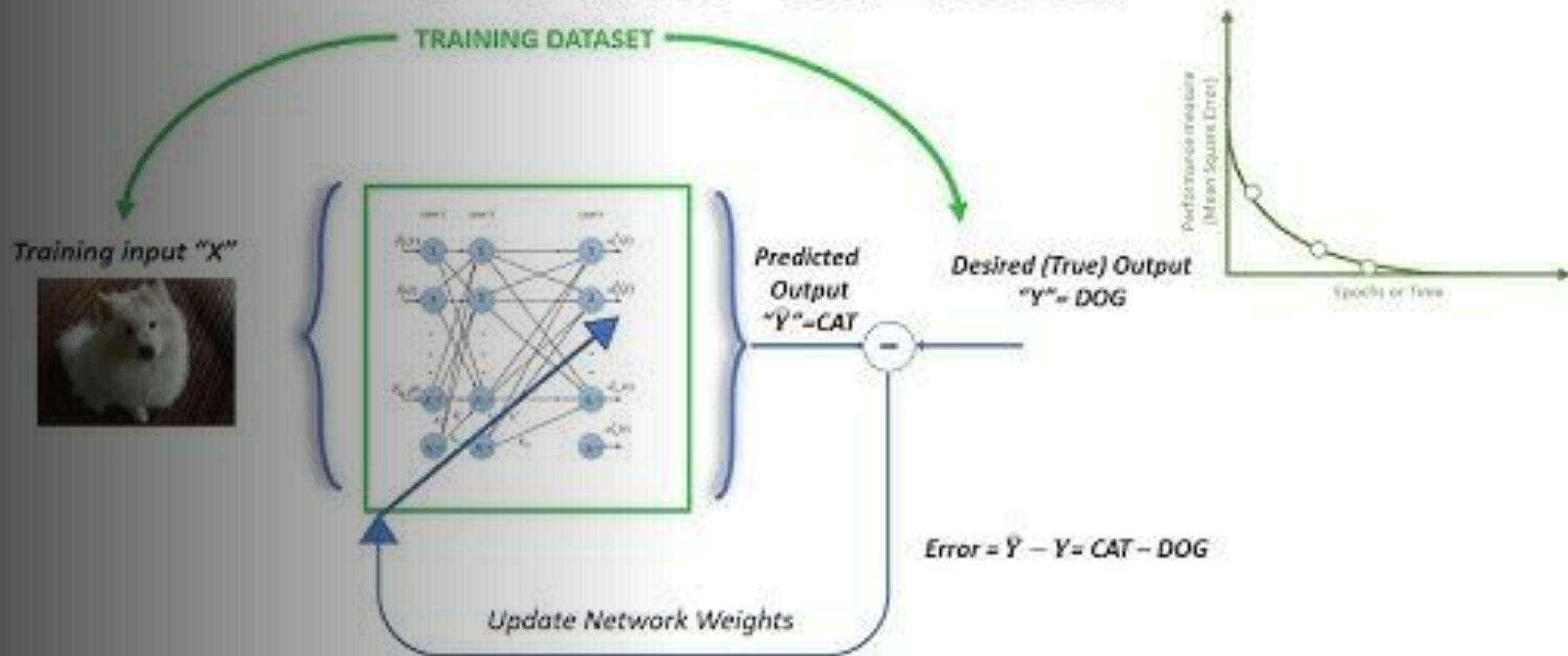
Input Layer

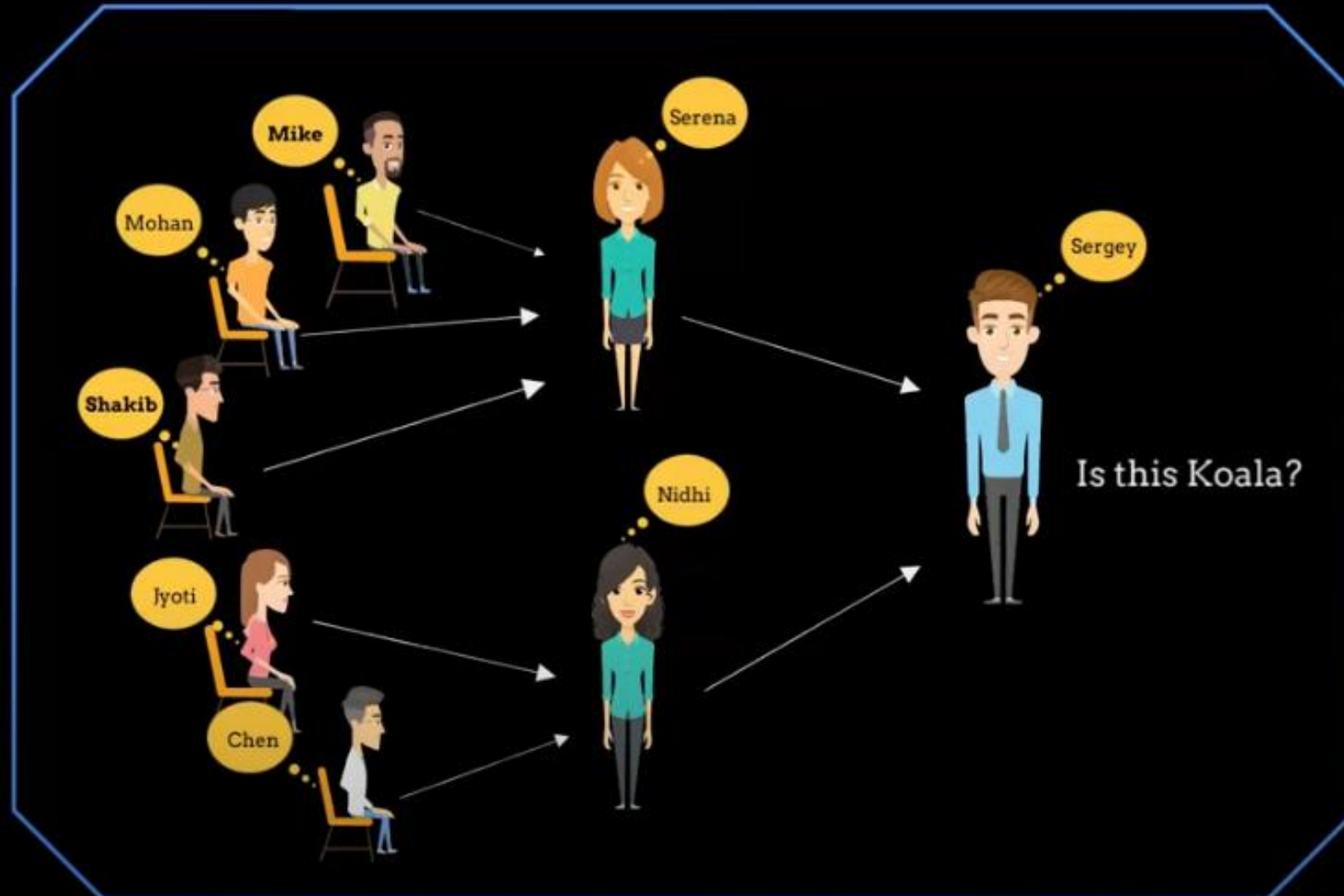
Hidden Layer

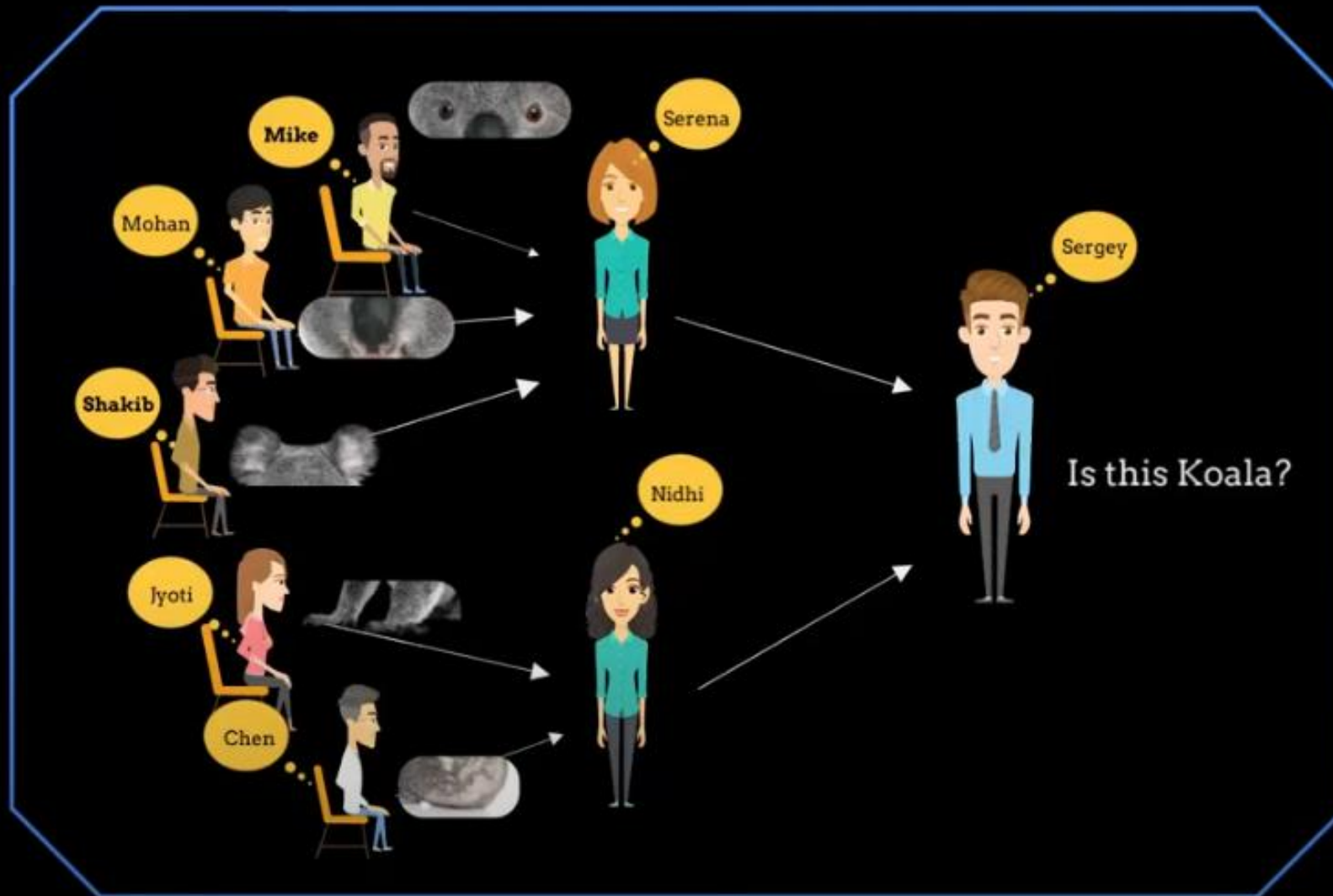
Output Layer



ARTIFICIAL NEURAL NETWORKS TRAINING PROCESS





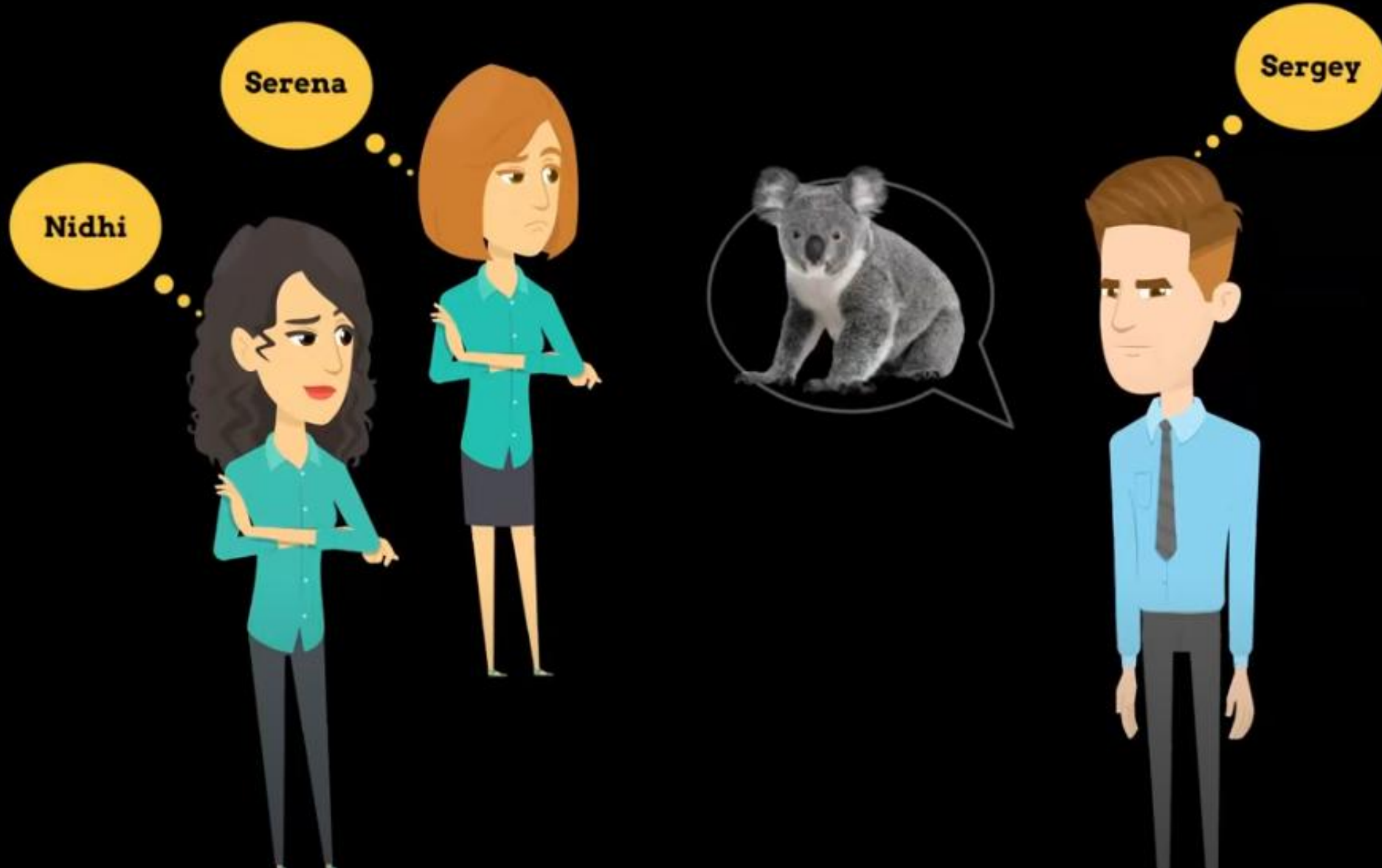


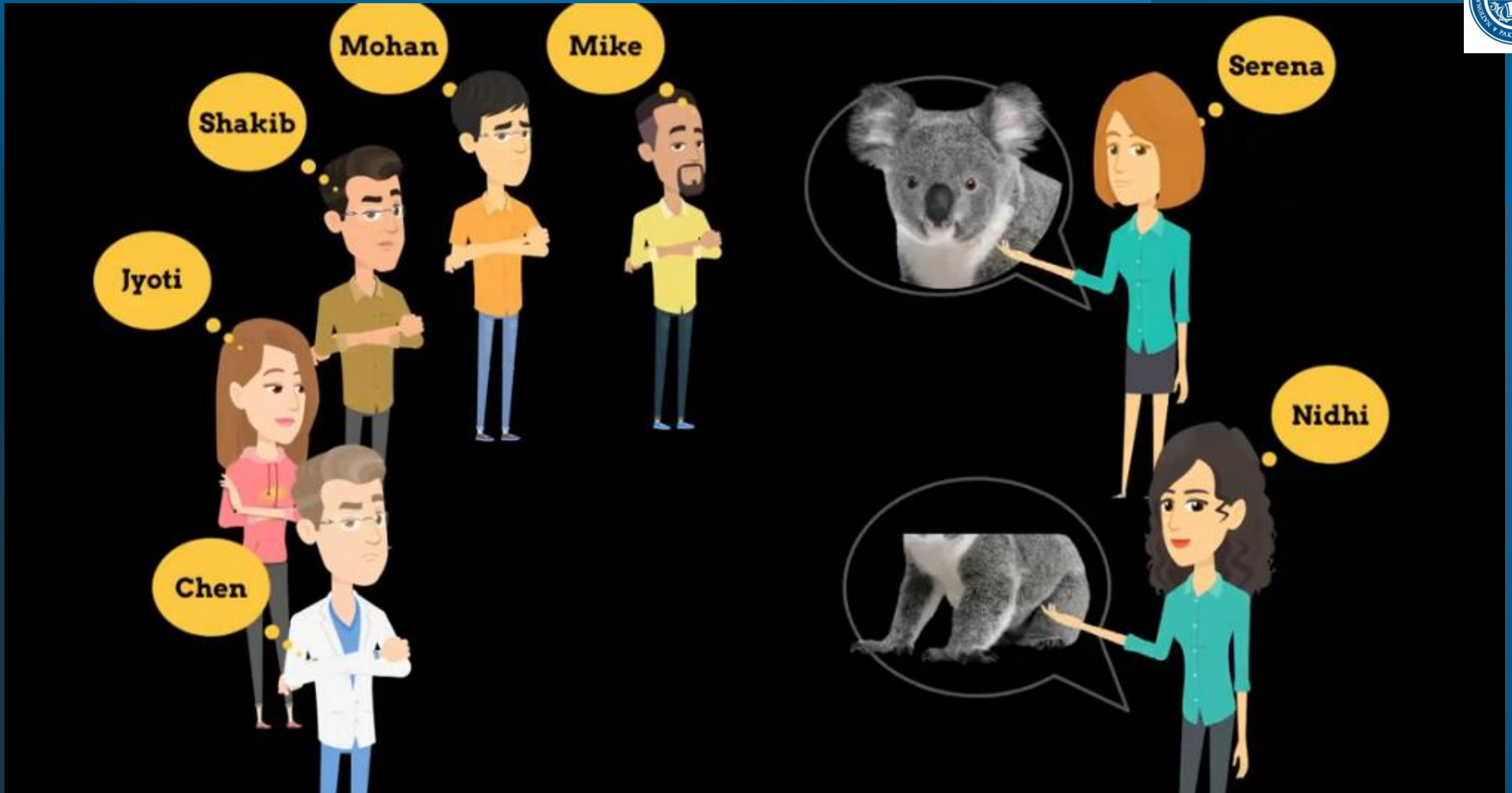
Sergey



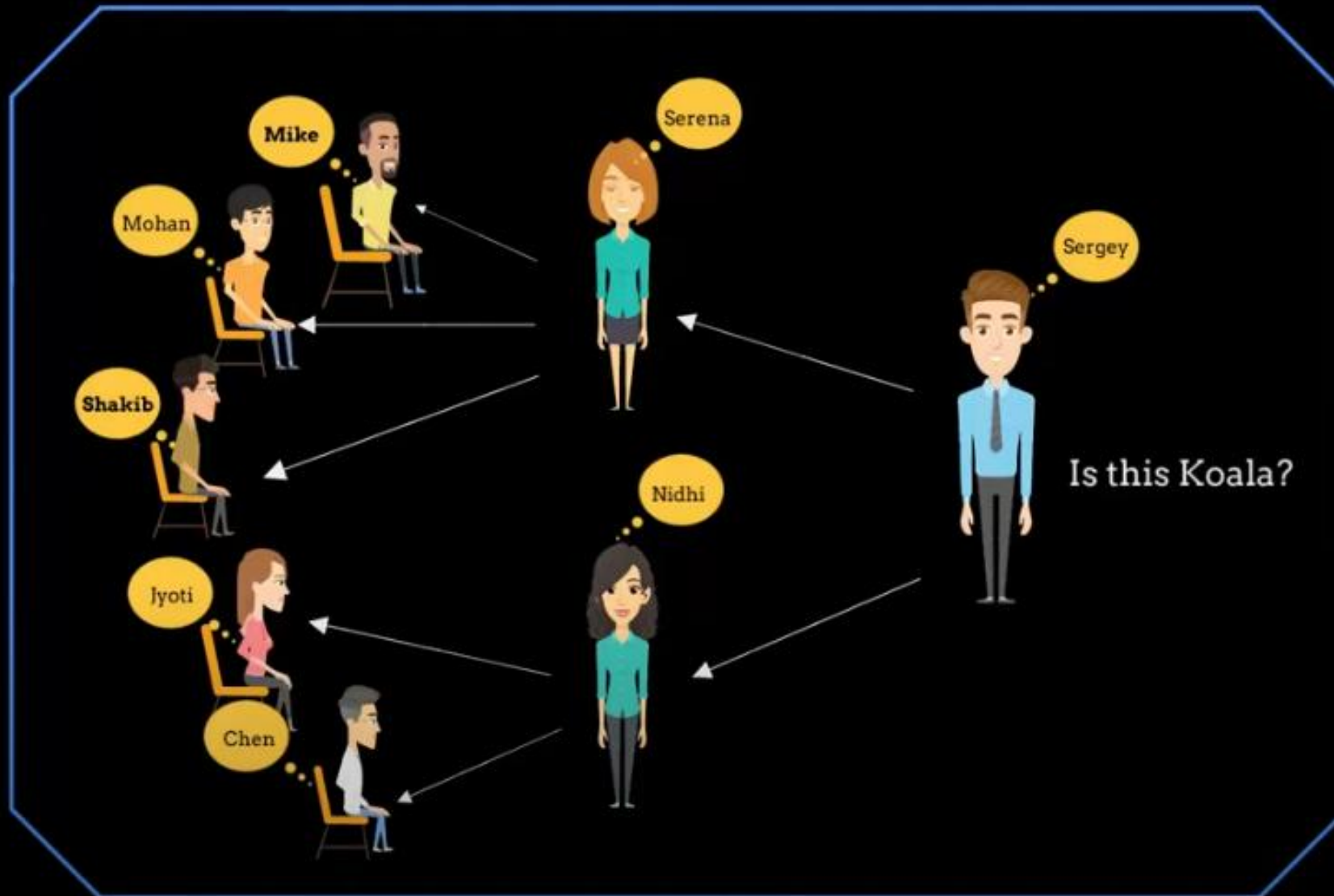
Supervisor

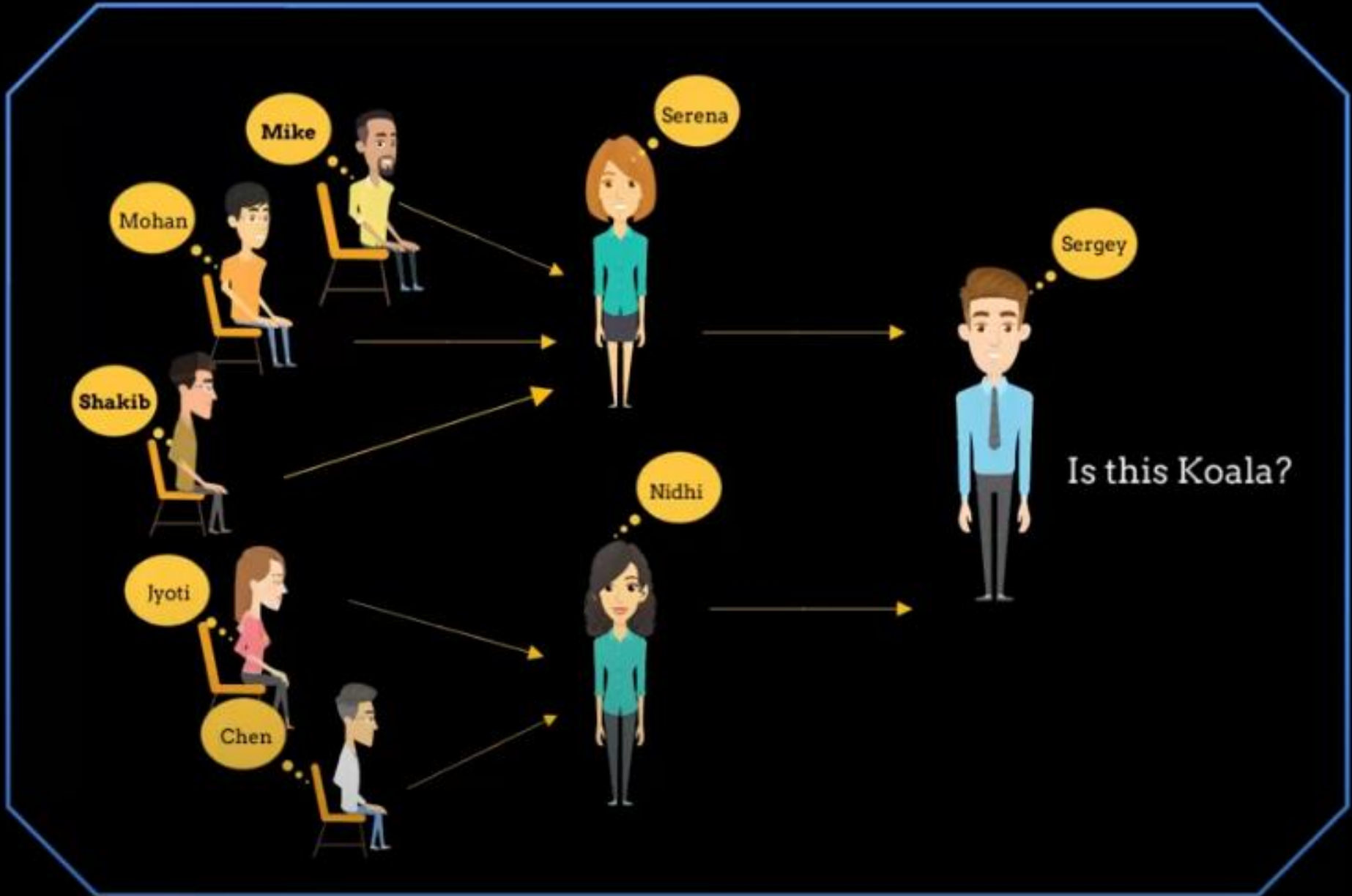




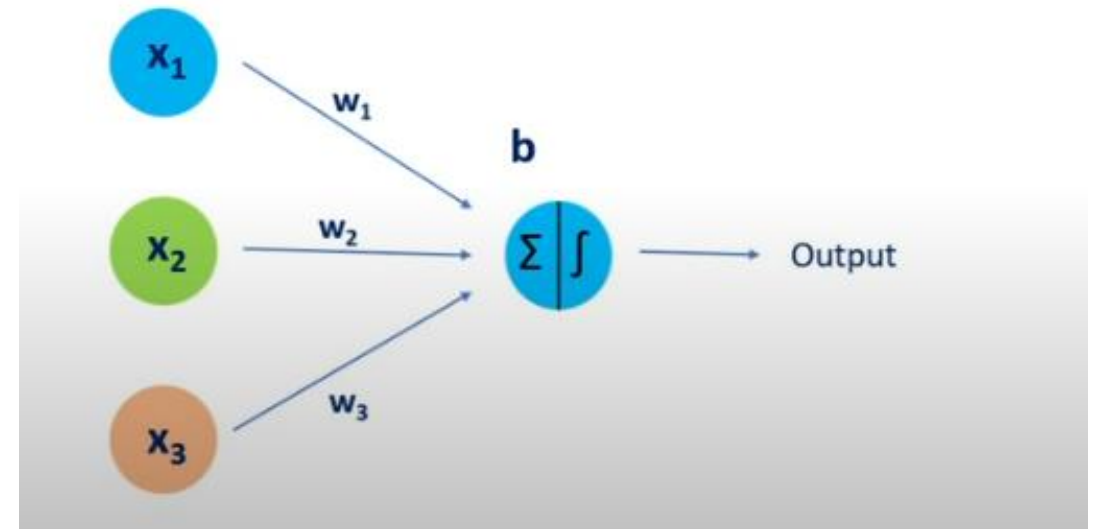
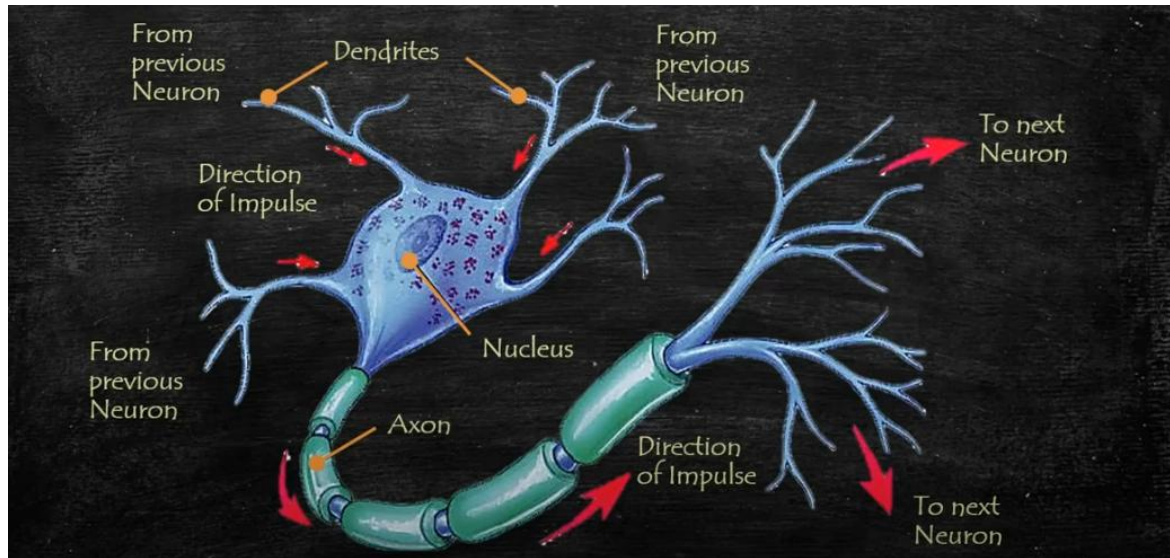


Backward Error Propagation



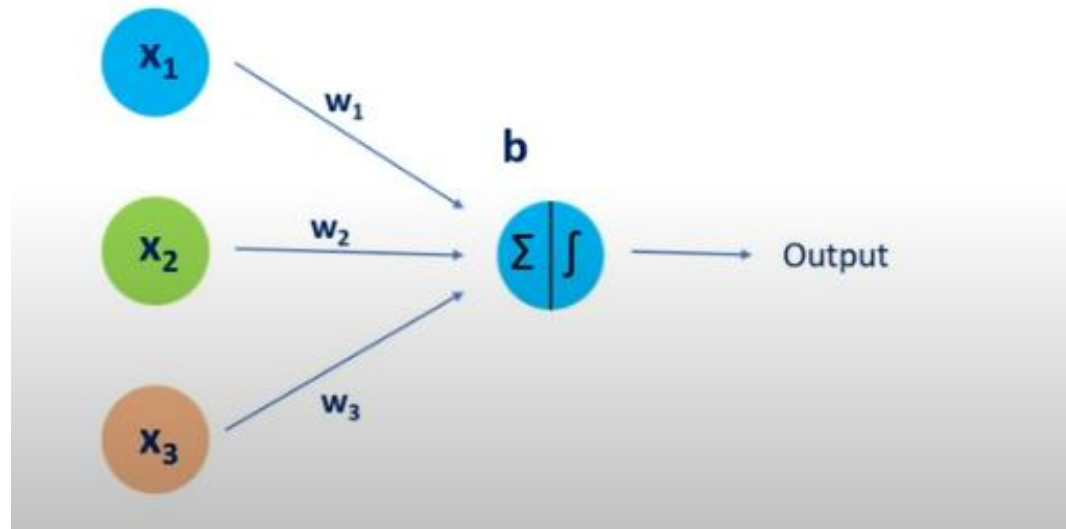


Perceptron?



A perceptron is a basic artificial neuron that takes inputs, applies weights, combines them, and produces an output using an activation function. It is used for tasks like binary classification and is a building block of neural networks

Perceptron?

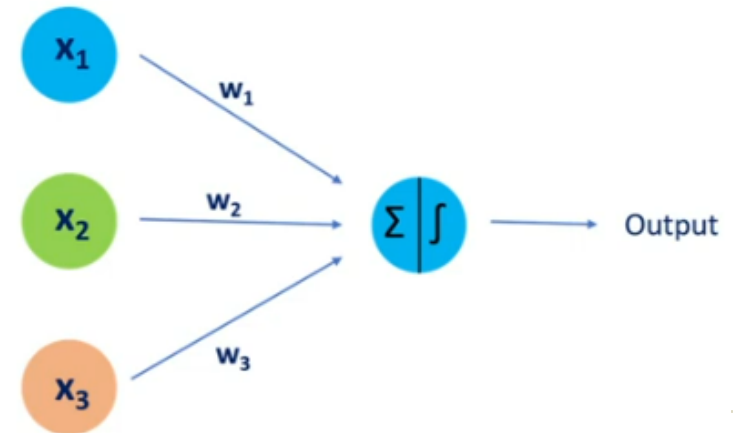


- Inputs
- Weights
- Bias
- Weighted Sum
- Activation Function
- Output

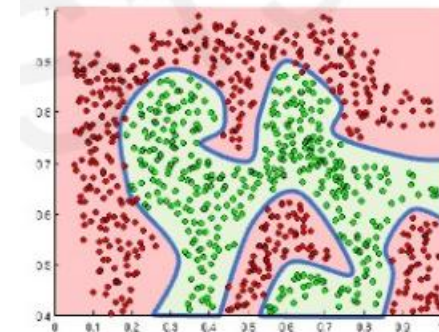
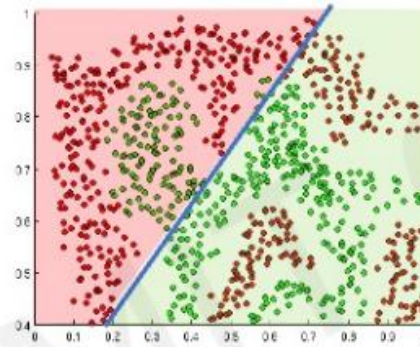
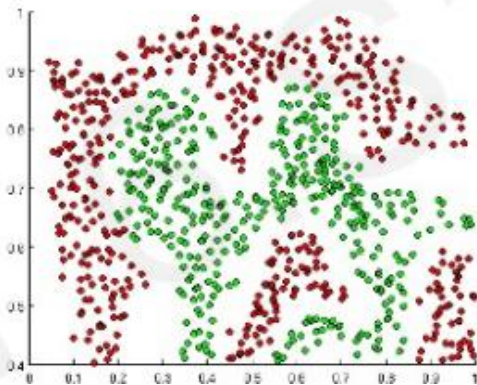
A perceptron is a basic artificial neuron that takes inputs, applies weights, combines them, and produces an output using an activation function. It is used for tasks like binary classification and is a building block of neural networks

Mathematical Representation of Perceptron?

- Input vector: $x = [x_1, x_2, \dots, x_n]$
- Weight vector: $w = [w_1, w_2, \dots, w_n]$
- Bias: b
- Summation function(Σ): $z = (w_1 * x_1) + (w_2 * x_2) + \dots + (w_n * x_n) + b$
- Activation function: $\phi(z)$



Activation Function in Perceptron?

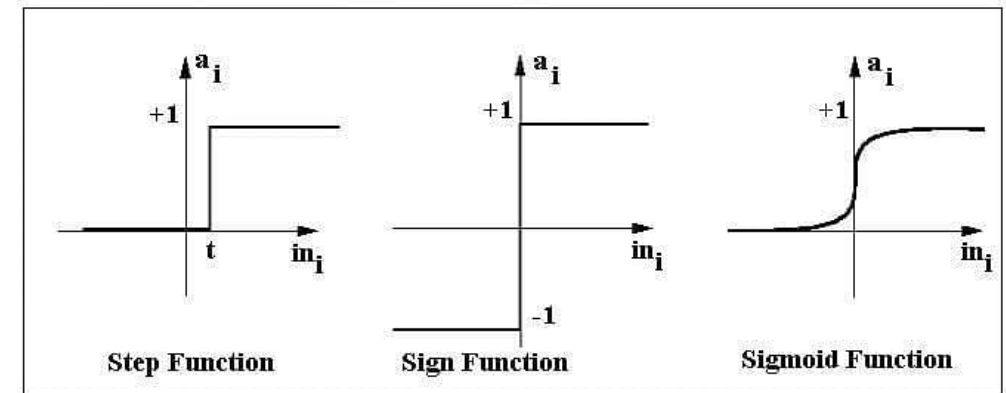


Non-linearities allow us to approximate arbitrarily complex functions

An activation function is a mathematical function applied to a neuron's processed input, transforming the neuron's output to the next layer in the network. The primary purpose of an activation function is to introduce non-linearity into the output of a neuron, which enables the network to learn complex patterns during training.

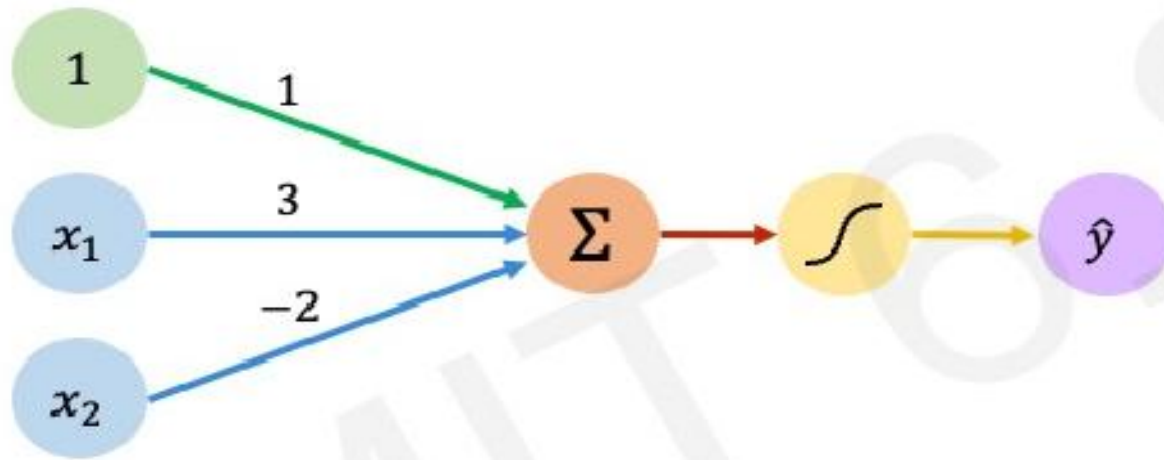
Activation Function in Perceptron?

- **Sigmoid function:** Output is a continuous value between 0 and 1.
Example: $\phi(z) = 1 / (1 + \exp(-z))$.
- **Step function:** Output is binary (0 or 1) based on a threshold. Example:
 $\phi(z) = 1$ if $z \geq 0$, else $\phi(z) = 0$.
- **Sign function:** Output is binary (-1 or 1) based on the sign of the input.
Example: $\phi(z) = 1$ if $z \geq 0$, else $\phi(z) = -1$.



An activation function is a mathematical function applied to a neuron's processed input, transforming the neuron's output to the next layer in the network. The primary purpose of an activation function is to introduce non-linearity into the output of a neuron, which enables the network to learn complex patterns during training.

The Perceptron: Example

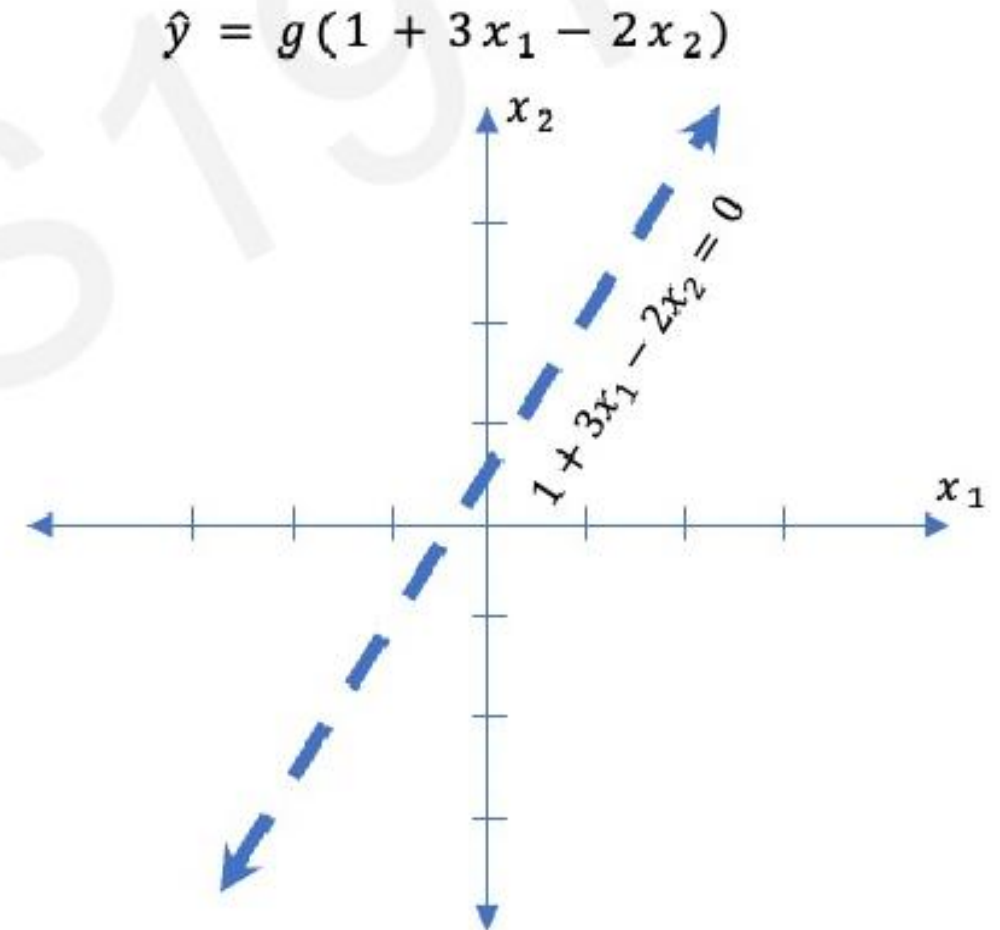
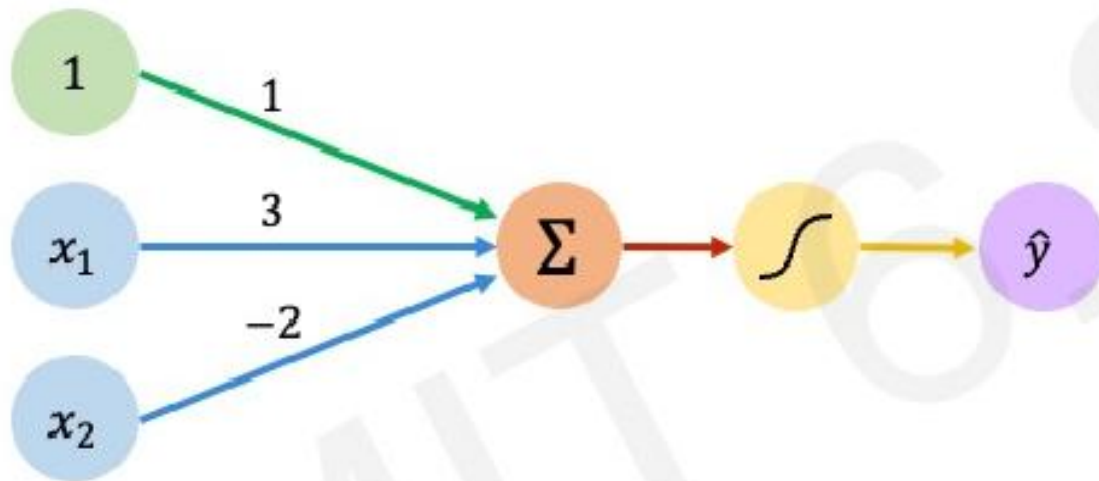


We have: $b = 1$ and $\mathbf{w} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

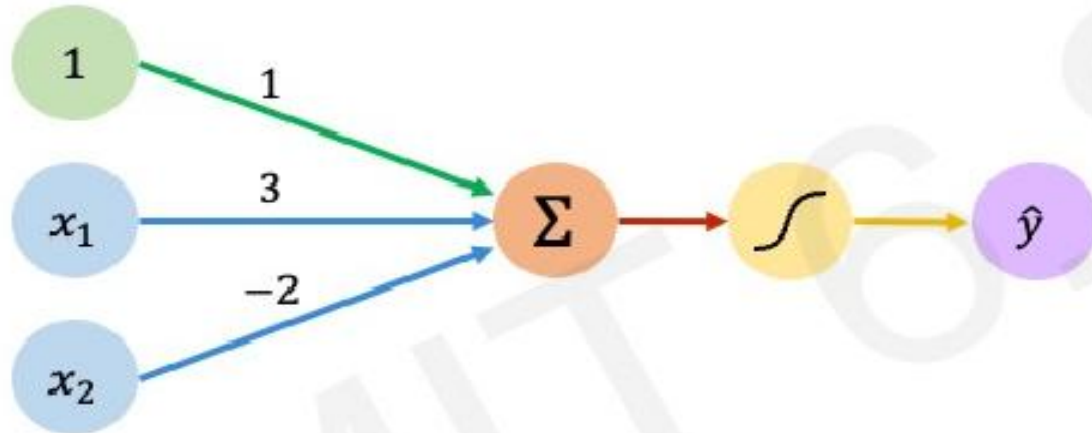
$$\begin{aligned}\hat{y} &= g(b + \mathbf{X}^T \mathbf{w}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(\underbrace{1 + 3x_1 - 2x_2})\end{aligned}$$

This is just a line in 2D!

The Perceptron: Example

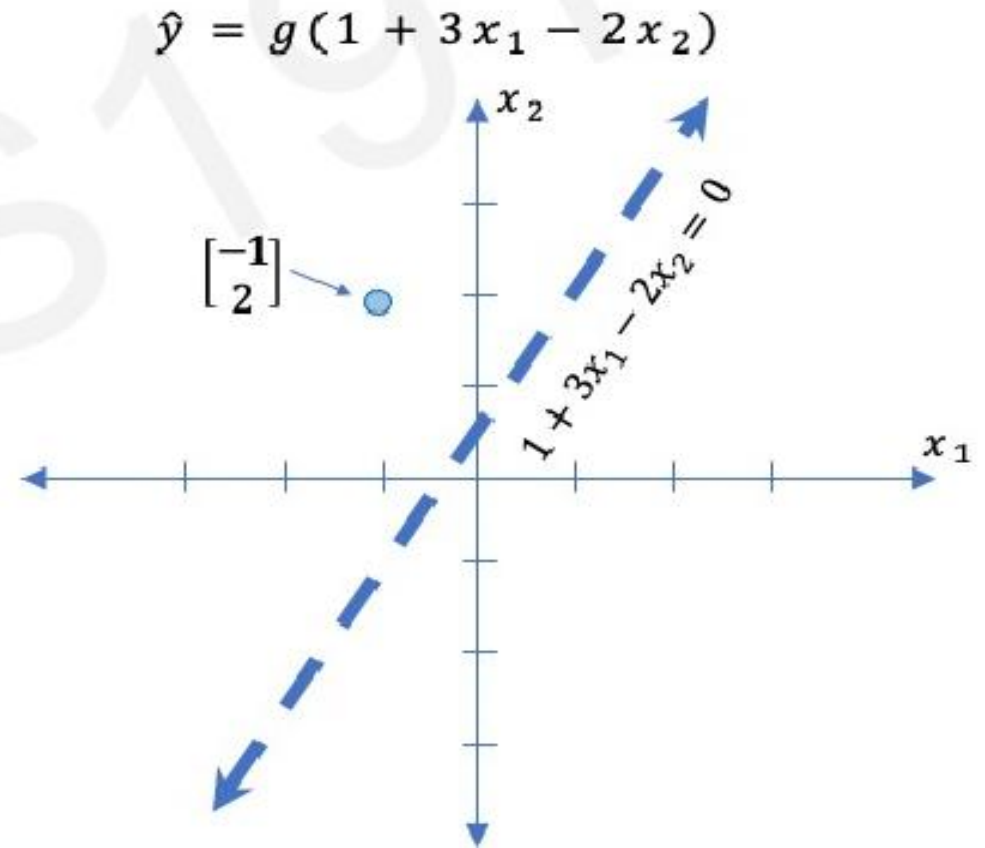


The Perceptron: Example

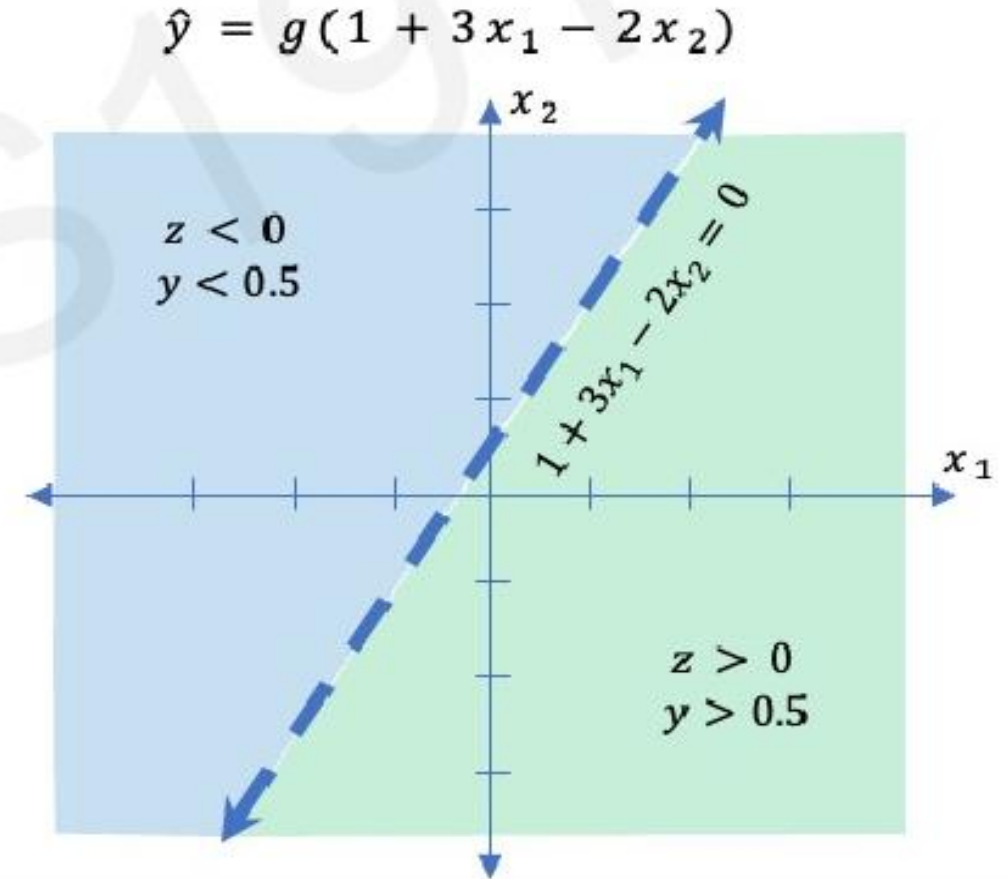
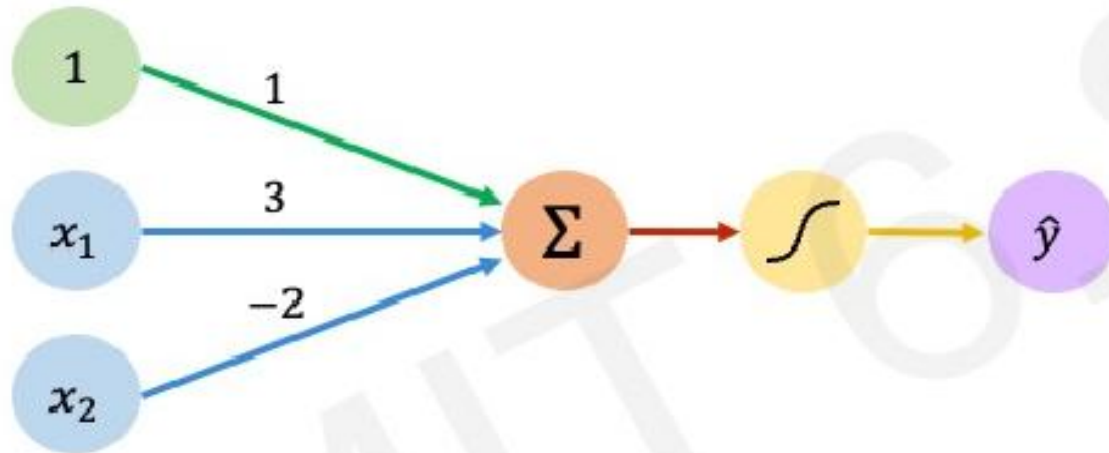


Assume we have input: $\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

$$\begin{aligned}\hat{y} &= g(1 + (3 * -1) - (2 * 2)) \\ &= g(-6) \approx 0.002\end{aligned}$$

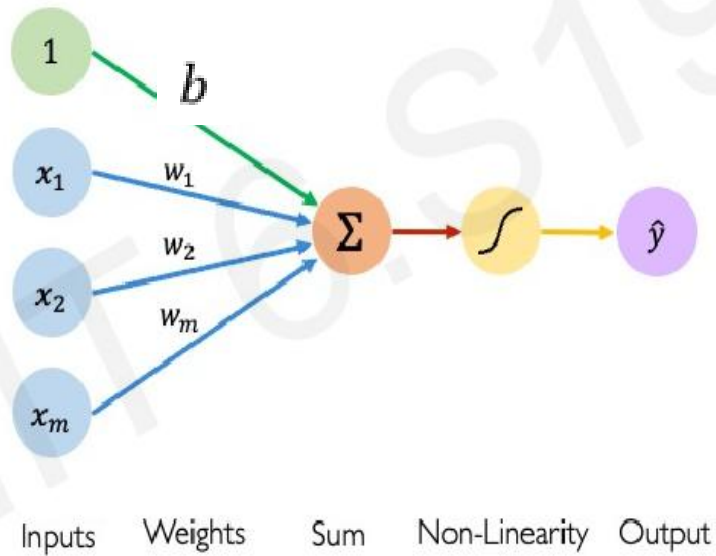


The Perceptron: Example

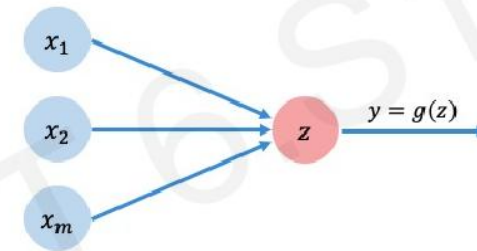


The Perceptron: Simplified

$$\hat{y} = g(b + \mathbf{X}^T \mathbf{W})$$



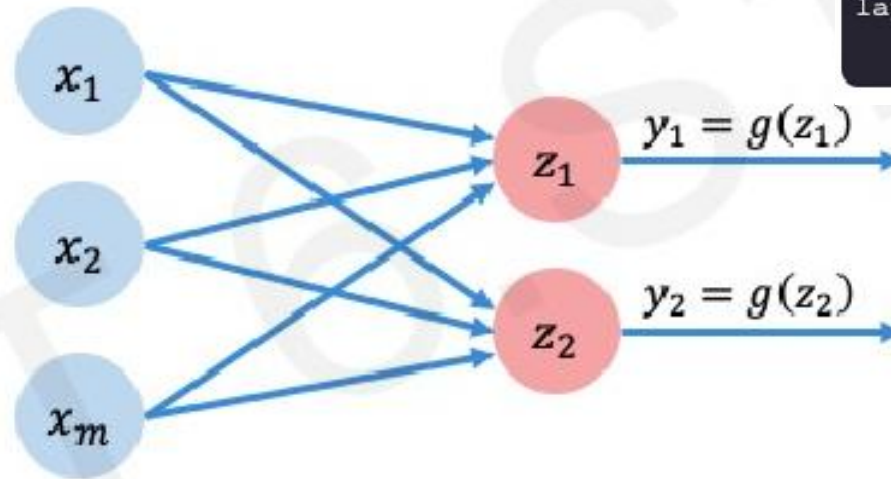
The Perceptron: Simplified



$$z = b + \sum_{j=1}^m x_j w_j$$

Multi Output Perceptron

Because all inputs are densely connected to all outputs, these layers are called **Dense** layers

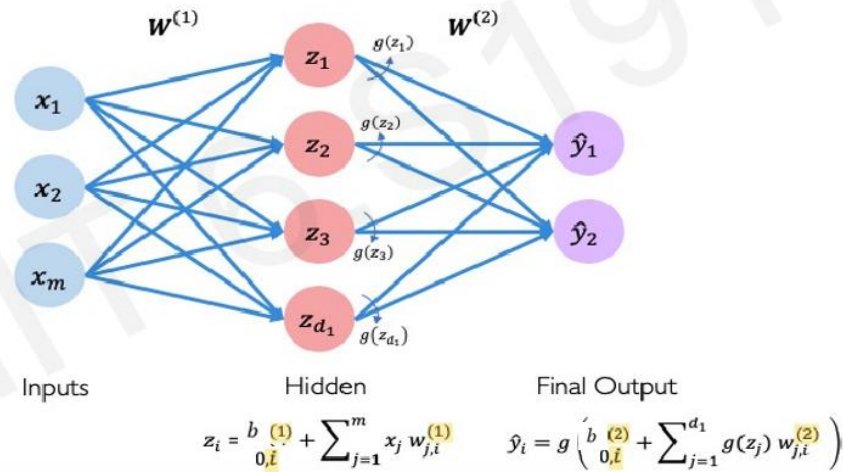


```
import torch.nn as nn  
  
layer = nn.Linear(in_features=m,  
                  out_features=2)
```

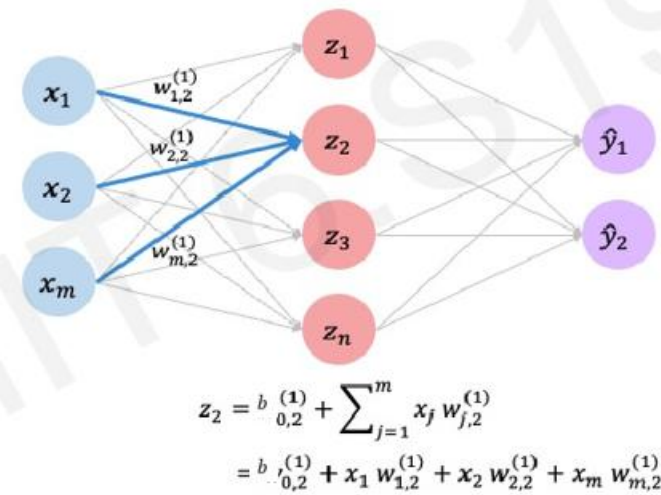
```
import tensorflow as tf  
  
layer = tf.keras.layers.Dense(  
    units=2)
```

$$z_i = b_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Single Layer Neural Network



Single Layer Neural Network



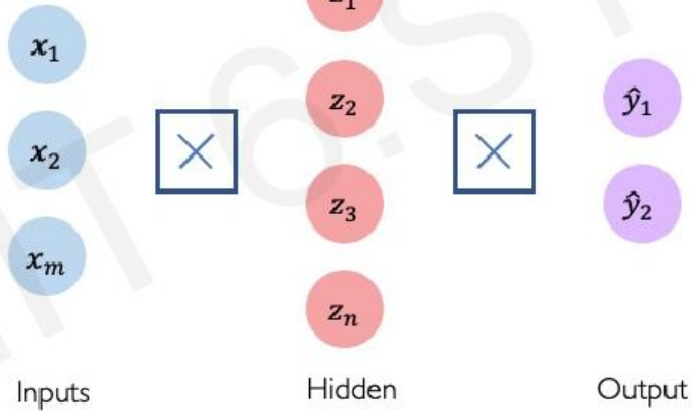
Multi Output Perceptron

```
import tensorflow as tf

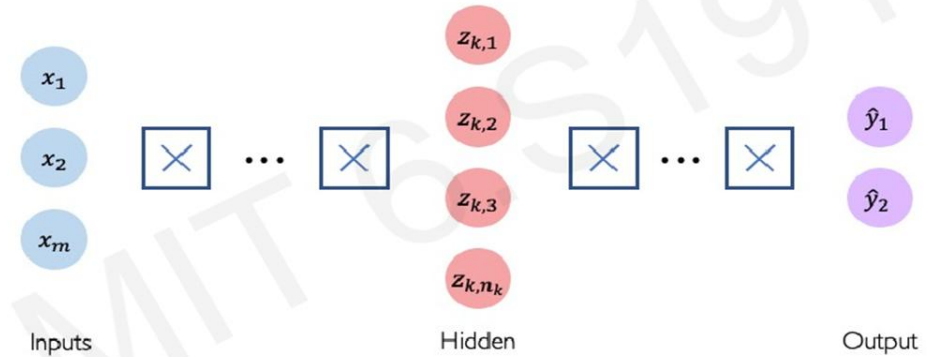
model = tf.keras.Sequential([
    tf.keras.layers.Dense(n),
    tf.keras.layers.Dense(2)
])
```

```
from torch import nn

model = nn.Sequential(
    nn.Linear(m, n),
    nn.ReLU(),
    nn.Linear(n, 2)
)
```



Deep Neural Network



$$z_{k,i} = b_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

Example Problem

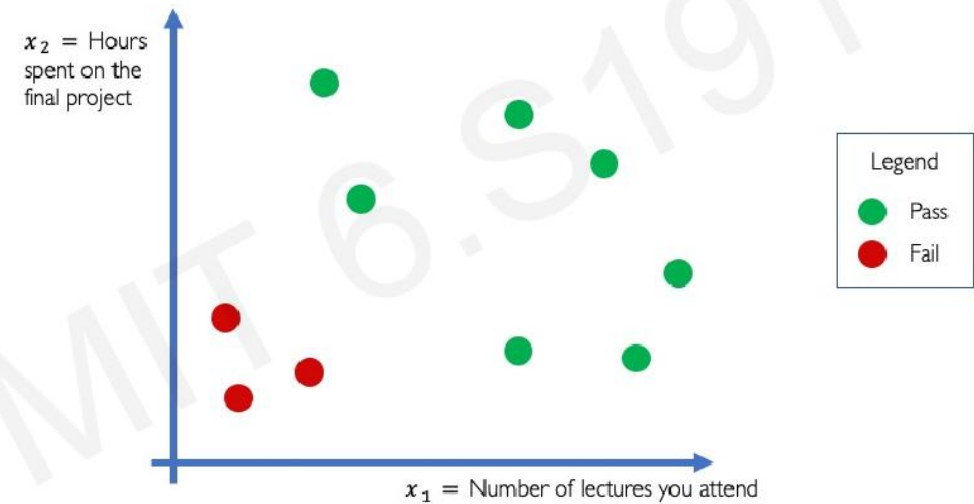
Will I pass this class?

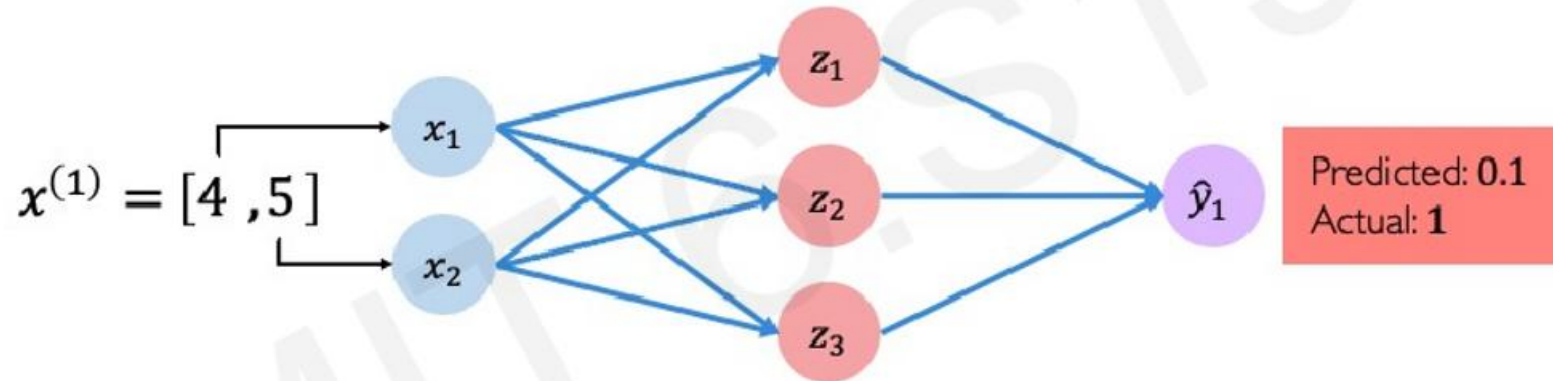
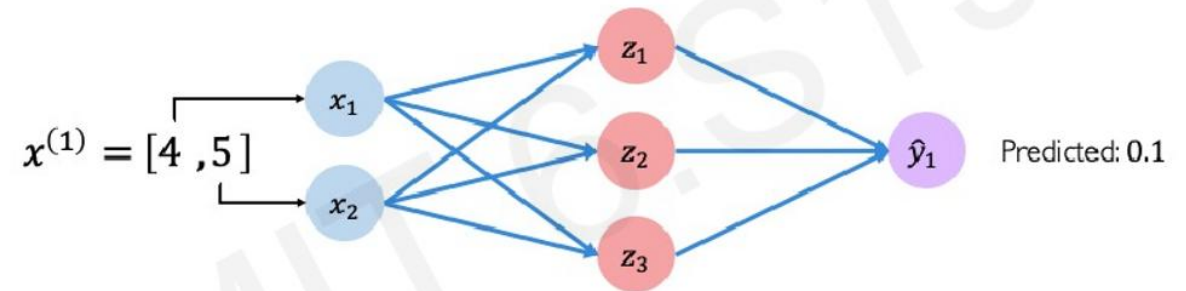
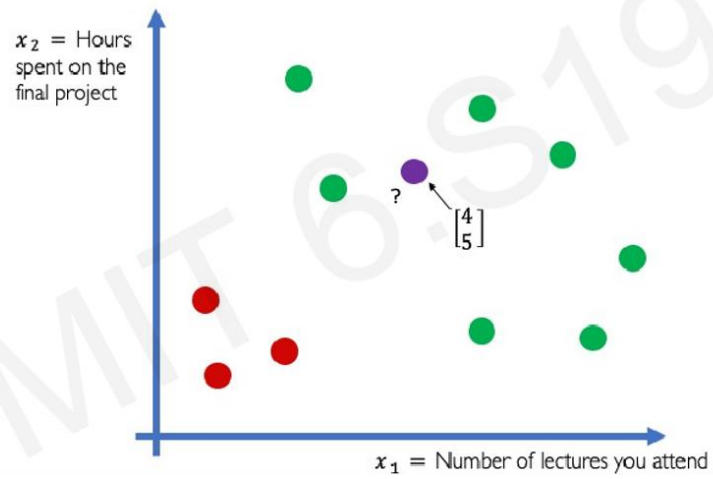
Let's start with a simple two feature model

x_1 = Number of lectures you attend

x_2 = Hours spent on the final project

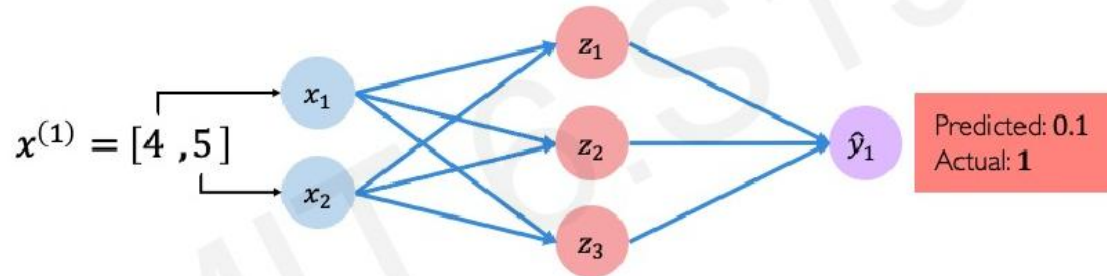
Example Problem: Will I pass this class?





Quantifying Loss

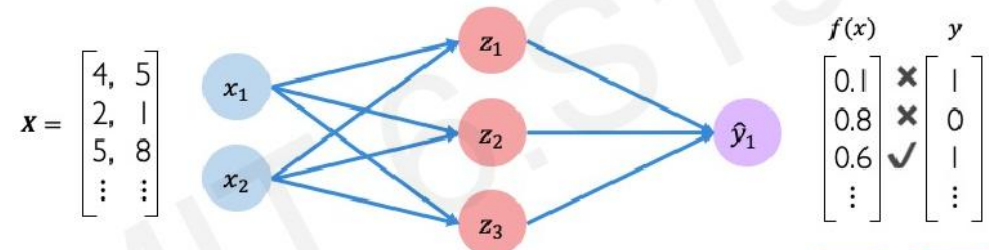
The **loss** of our network measures the cost incurred from incorrect predictions



$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset



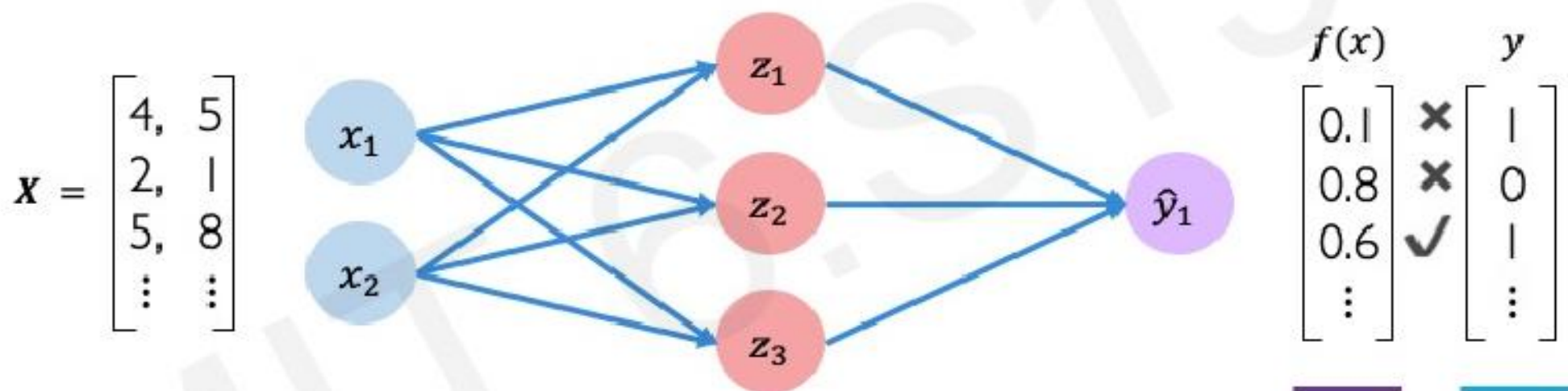
Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Binary Cross Entropy Loss

Cross entropy loss can be used with models that output a probability between 0 and 1



$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}}) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log(1 - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}})$$



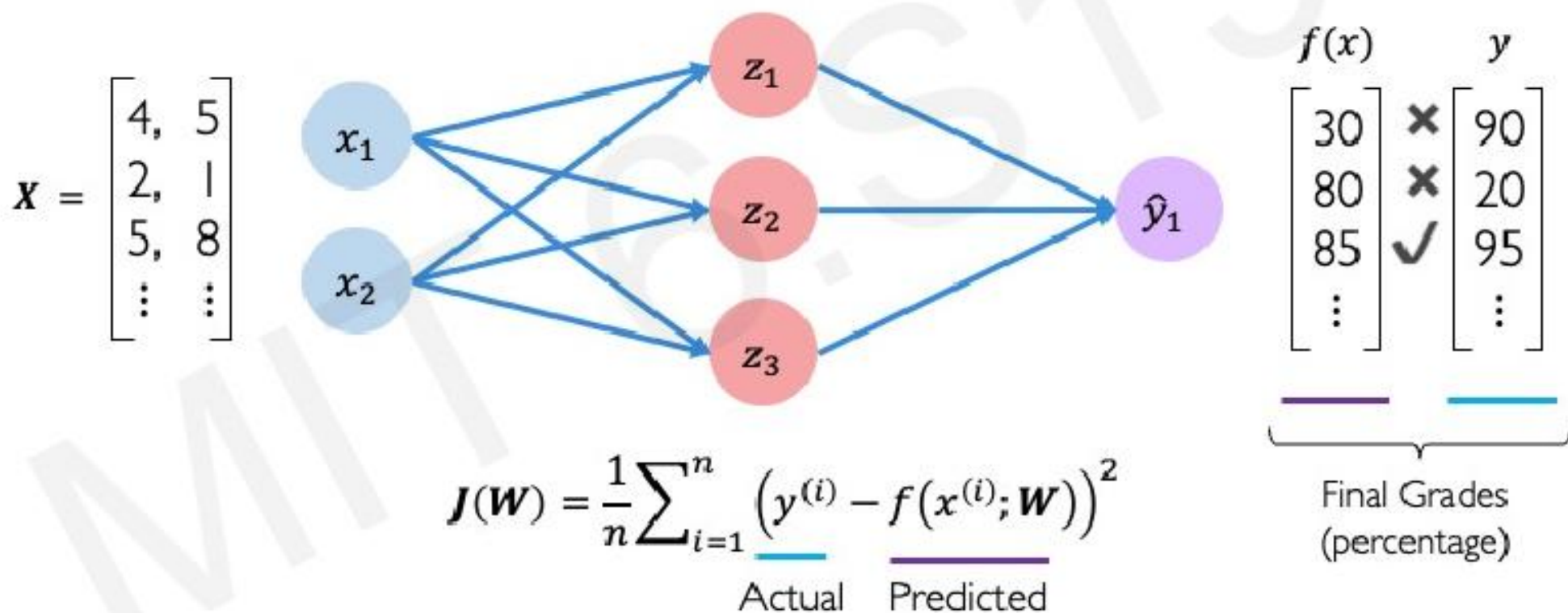
```
loss = tf.reduce_mean( tf.nn.softmax_cross_entropy_with_logits(y, predicted) )
```



```
loss = torch.nn.functional.cross_entropy( predicted, y )
```

Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers



```
loss = tf.reduce_mean( tf.square(tf.subtract(y, predicted)) )  
loss = tf.keras.losses.MSE( y, predicted )
```

```
loss = torch.nn.functional.mse_loss(  
    predicted, y )
```



Loss Optimization

We want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

Loss Optimization

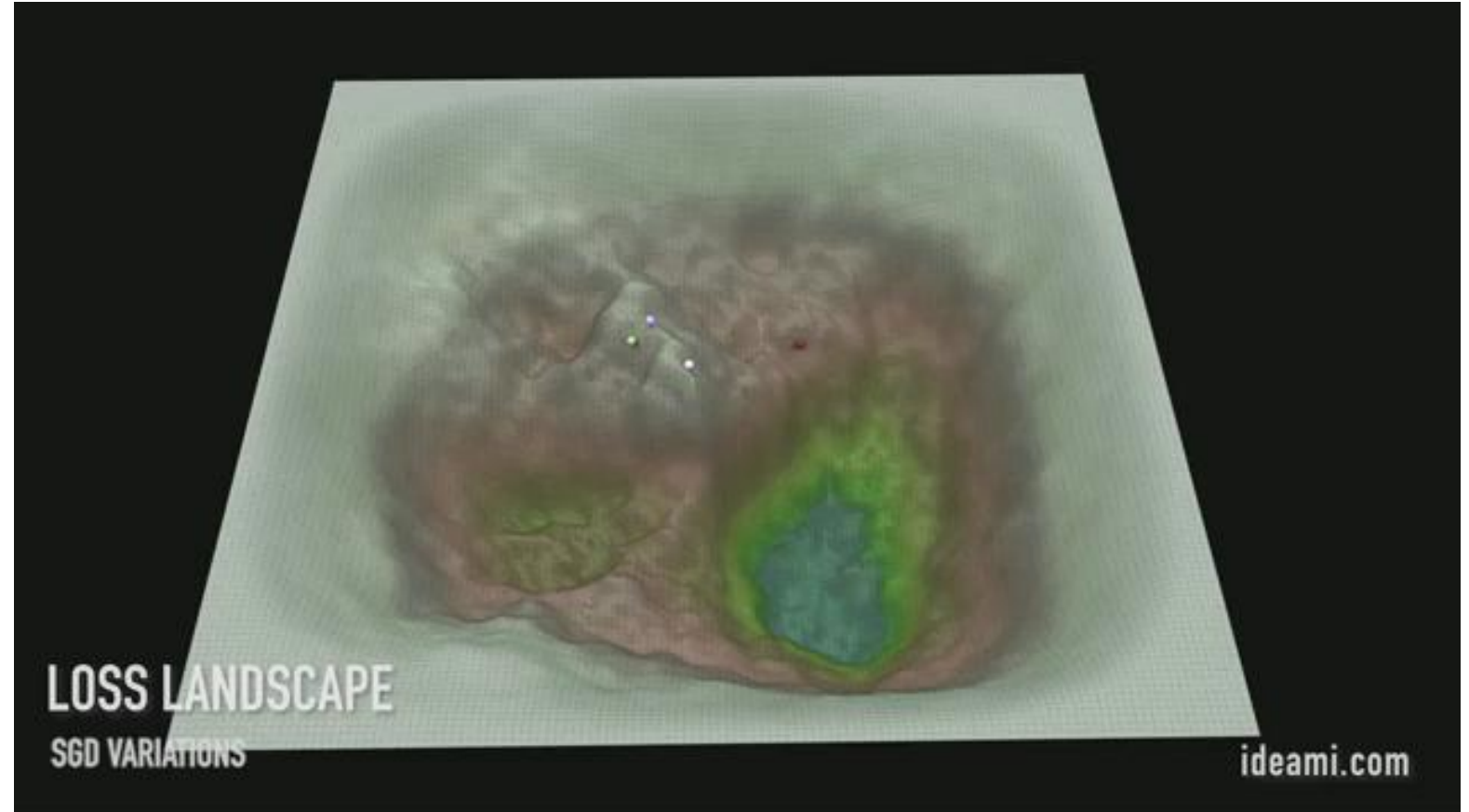
We want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

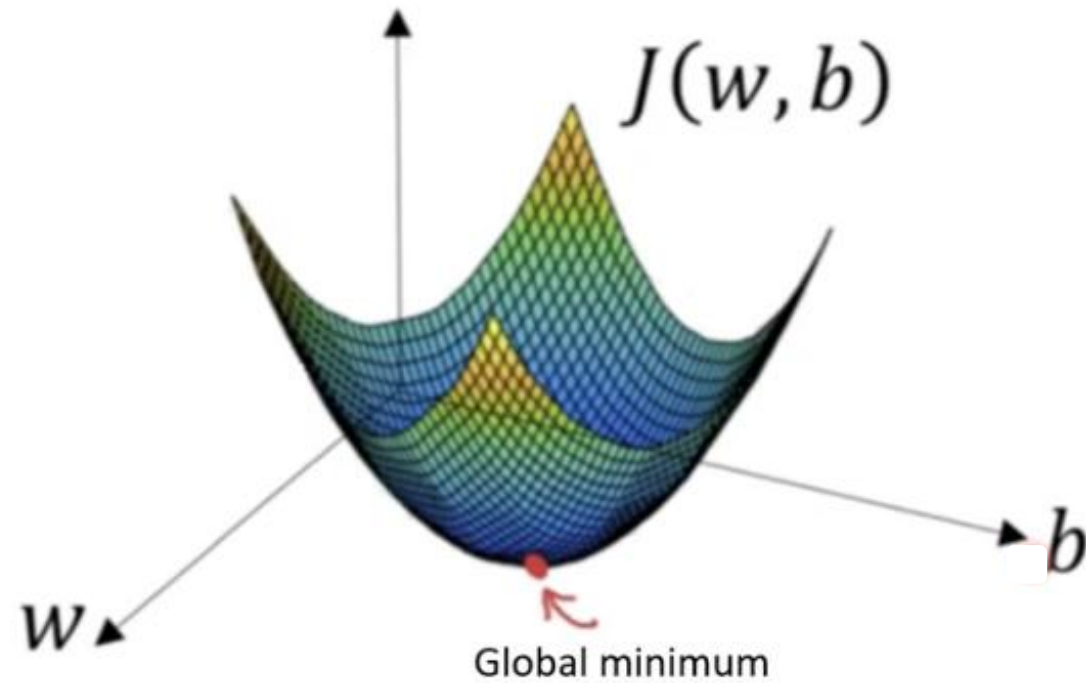
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

Remember:
 $\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$

Gradient Descent?



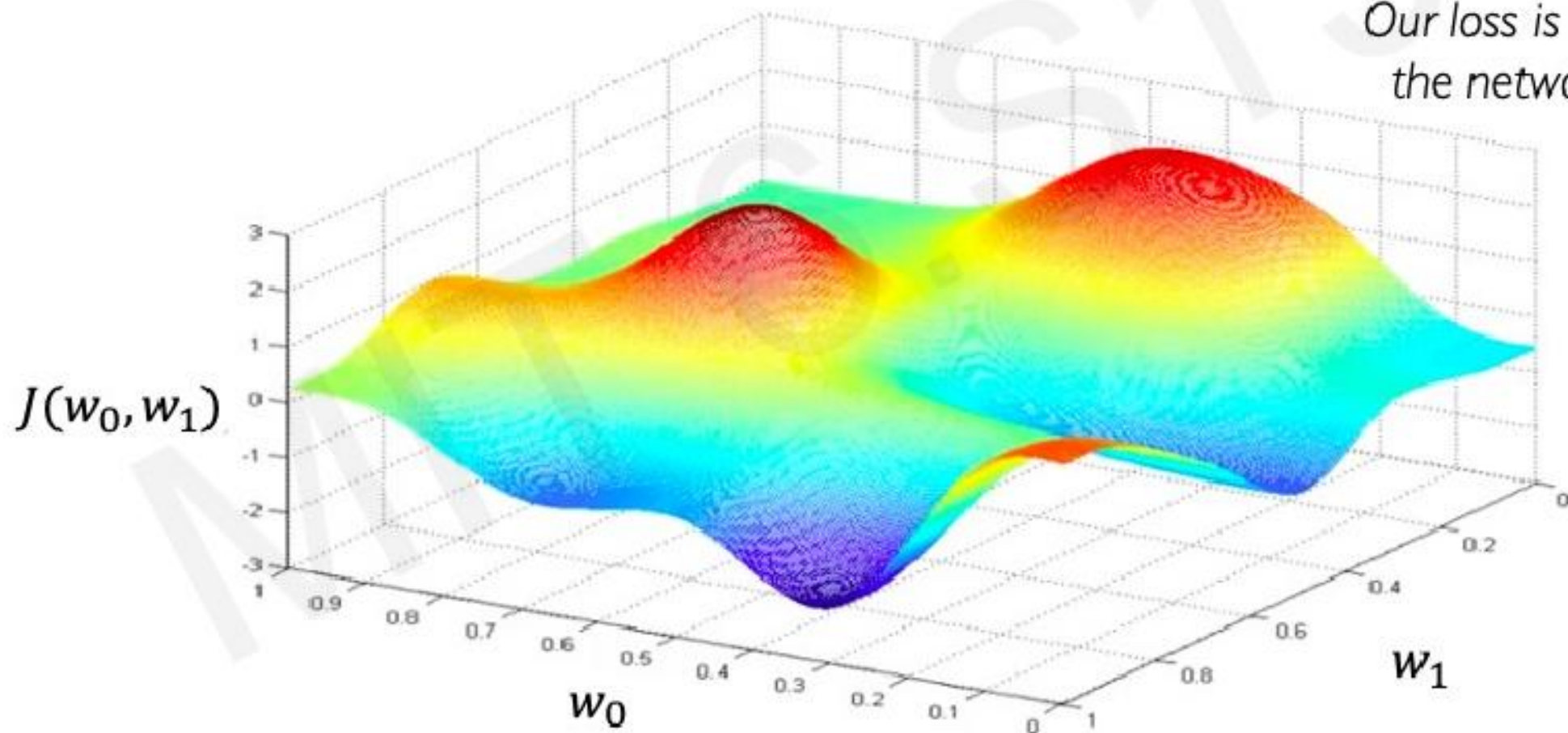
Gradient Descent in 3 Dimension?



Loss Optimization

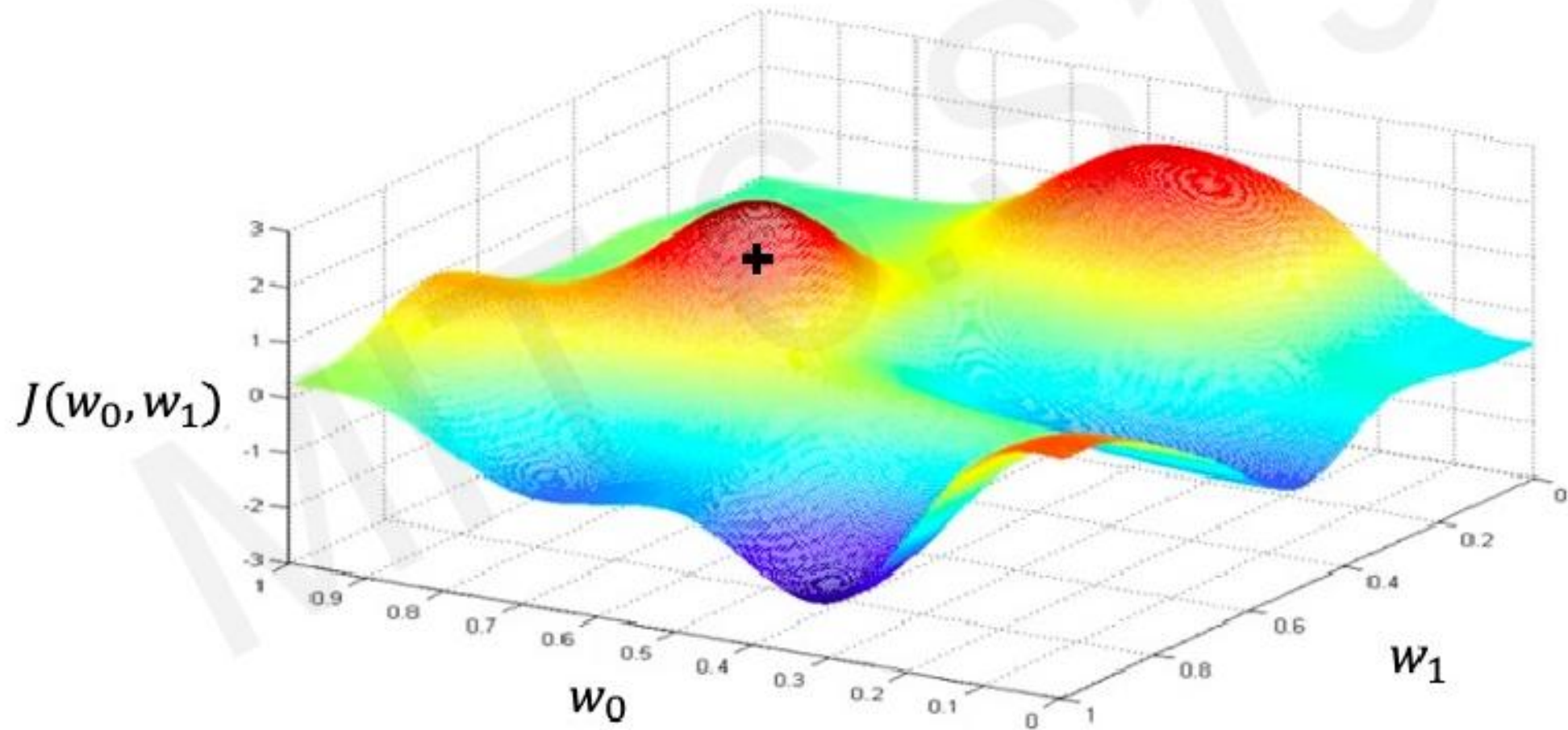
$$W^* = \underset{W}{\operatorname{argmin}} J(W)$$

Remember:
*Our loss is a function of
the network weights!*



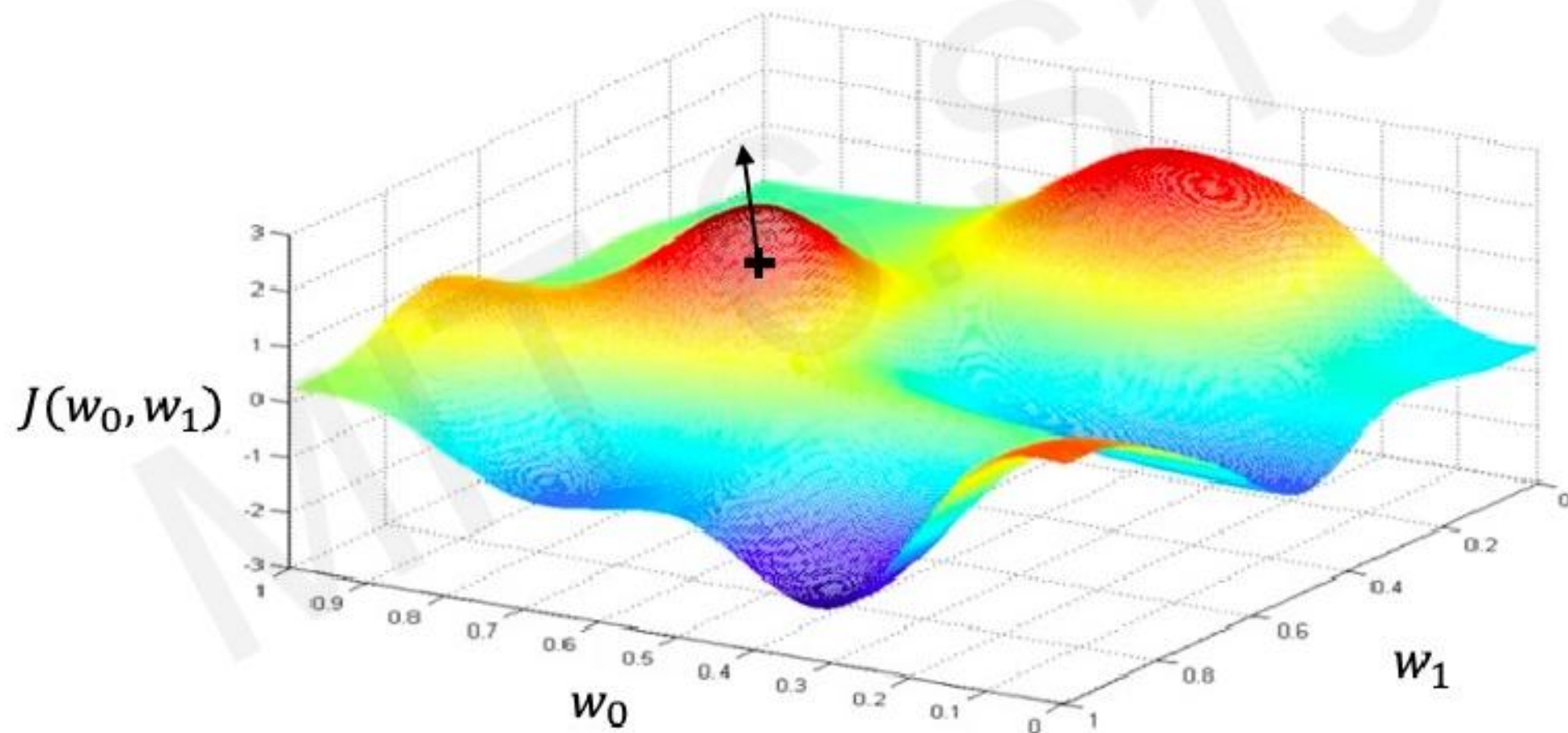
Loss Optimization

Randomly pick an initial (w_0, w_1)



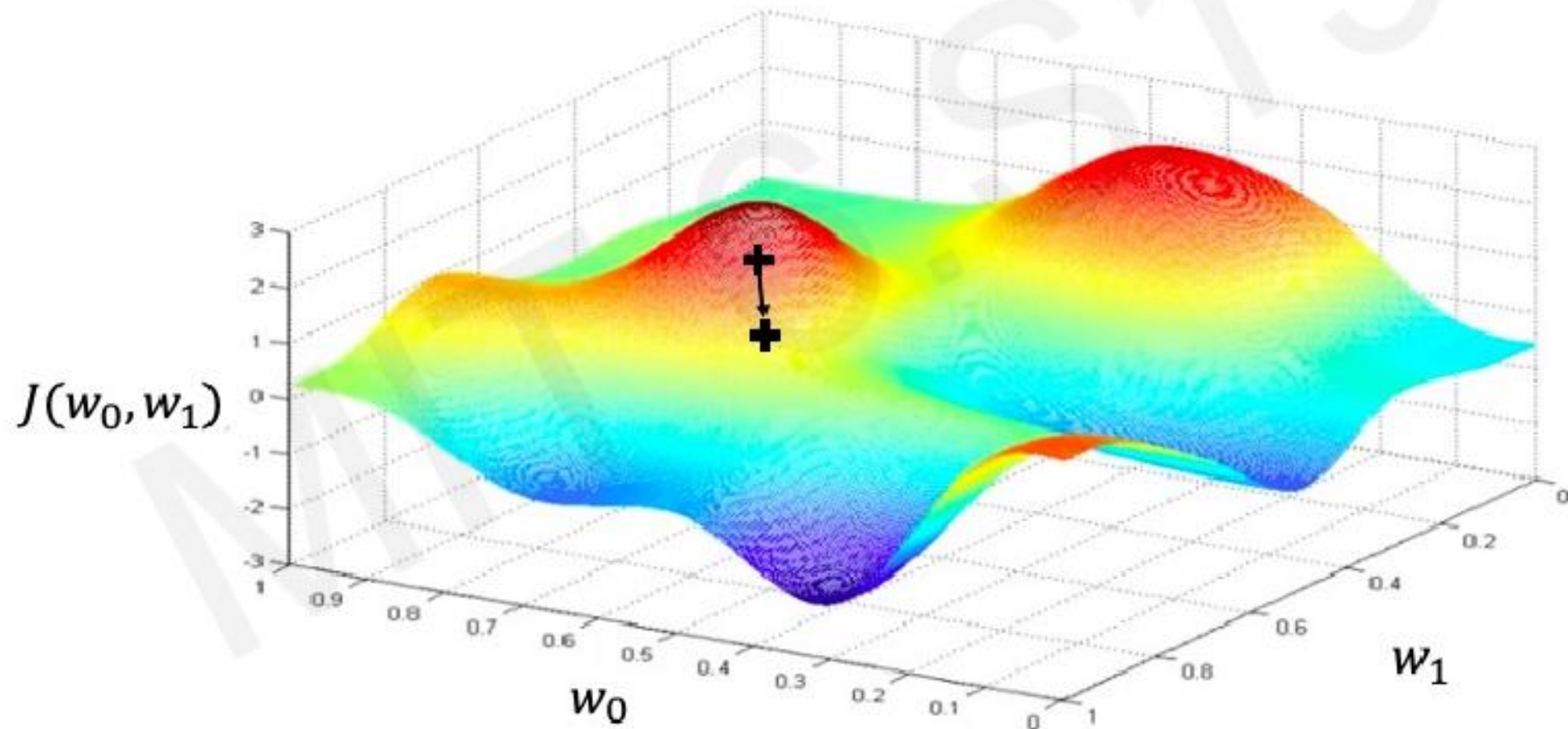
Loss Optimization

Compute gradient, $\frac{\partial J(W)}{\partial W}$



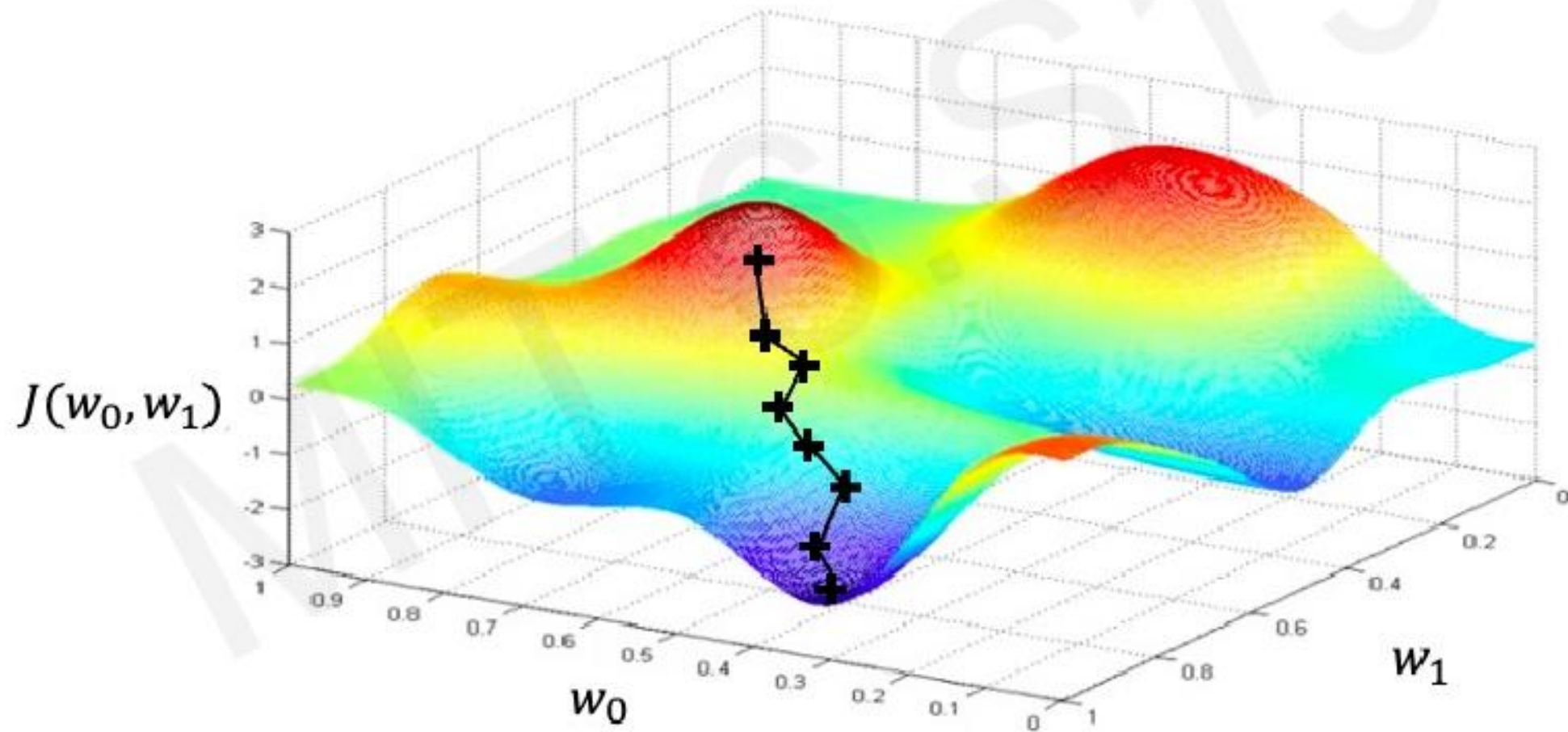
Loss Optimization

Take small step in opposite direction of gradient



Gradient Descent

Repeat until convergence



Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights



Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

    weights = weights - lr * gradient
```



Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

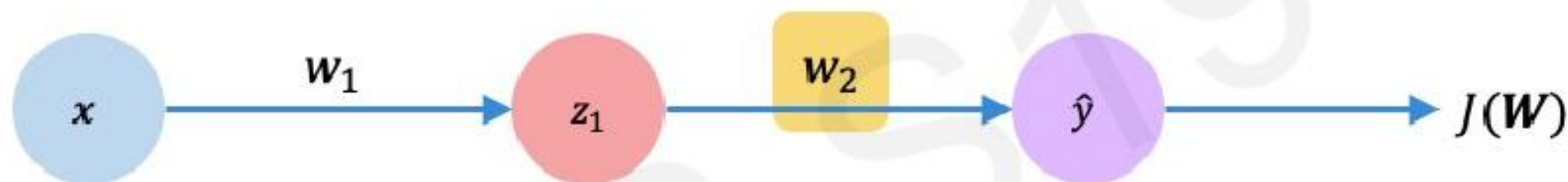
```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

    weights = weights - lr * gradient
```

Computing Gradients: Backpropagation



How does a small change in one weight (ex. w_2) affect the final loss $J(W)$?

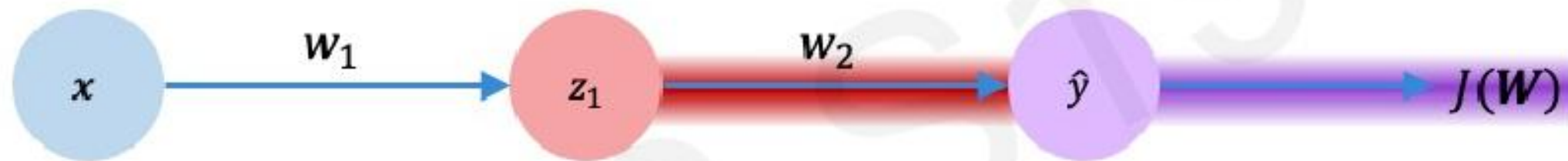
Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2} =$$

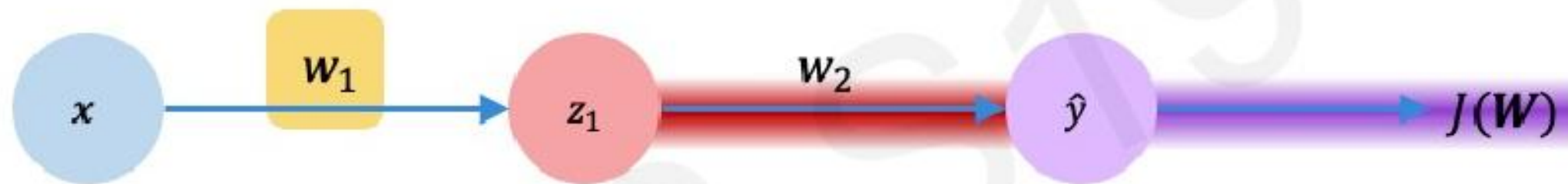
Let's use the chain rule!

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

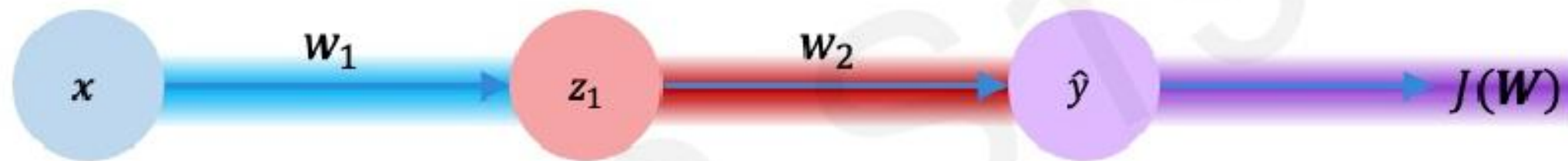
Apply chain rule! Apply chain rule!

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

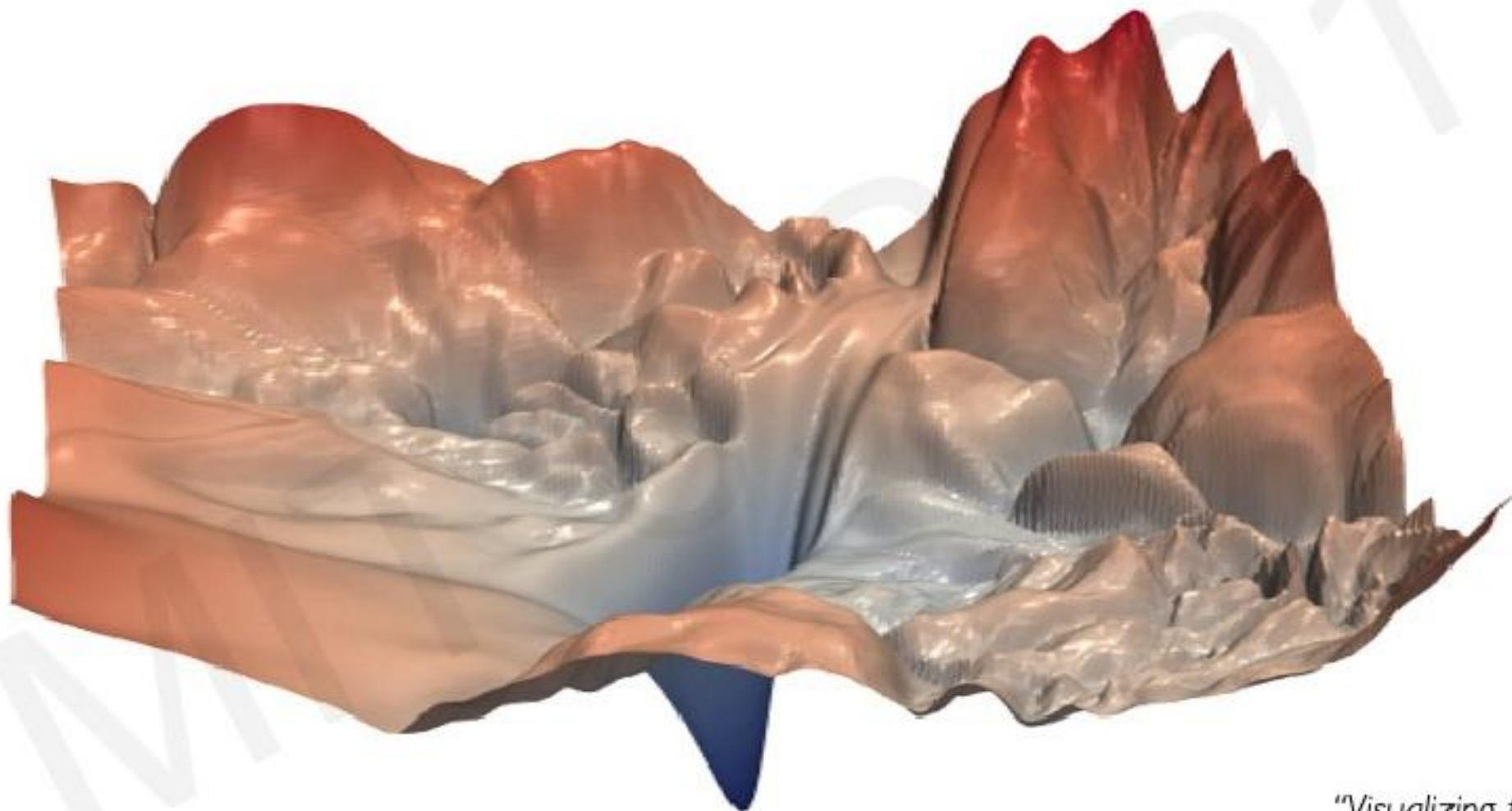
Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Repeat this for **every weight in the network** using gradients from later layers

Training Neural Networks is Difficult



"Visualizing the loss landscape of neural nets". Dec 2017.

Loss Functions Can Be Difficult to Optimize

Remember:

Optimization through gradient descent

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

Loss Functions Can Be Difficult to Optimize

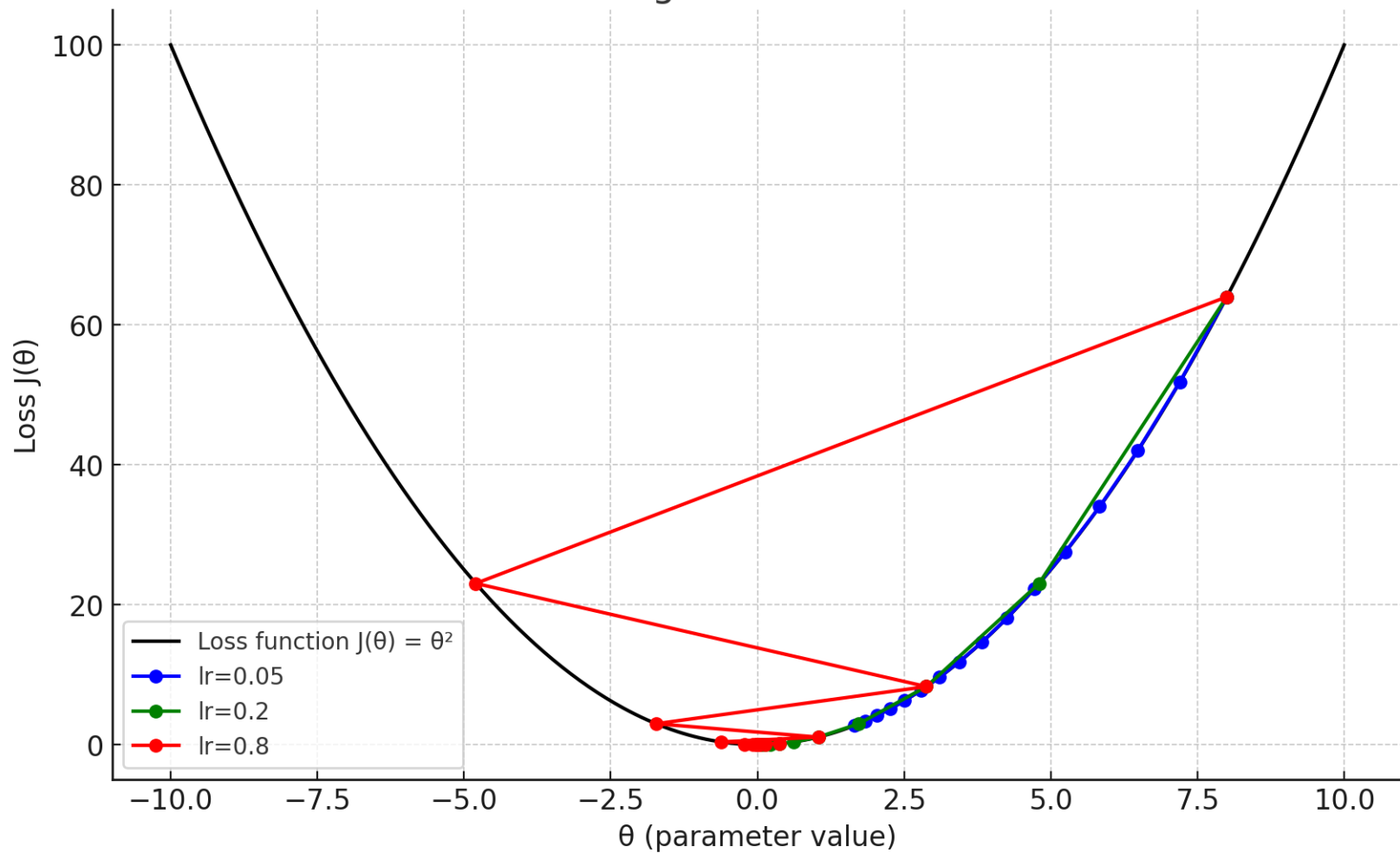
Remember:

Optimization through gradient descent

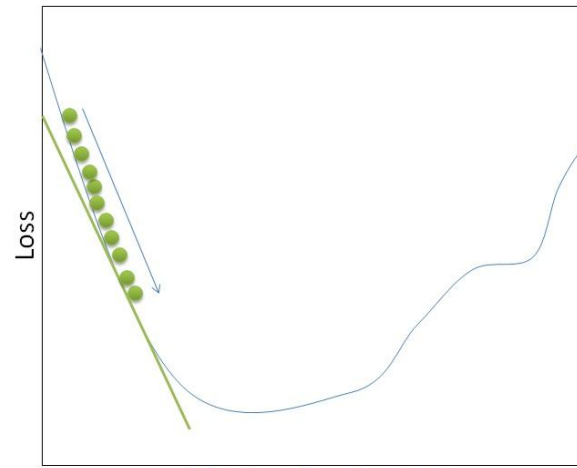
$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

How can we set the
learning rate?

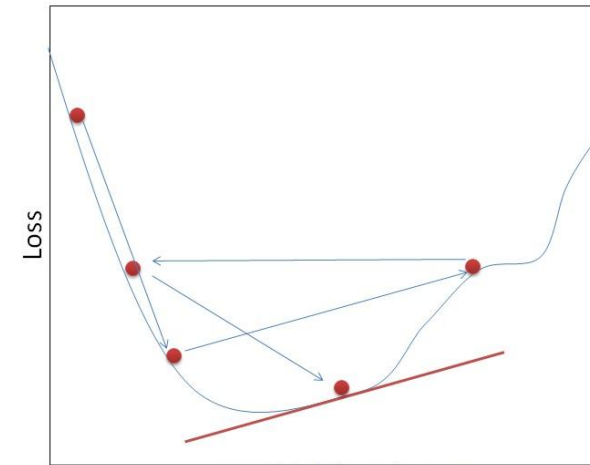
Effect of Learning Rate on Gradient Descent



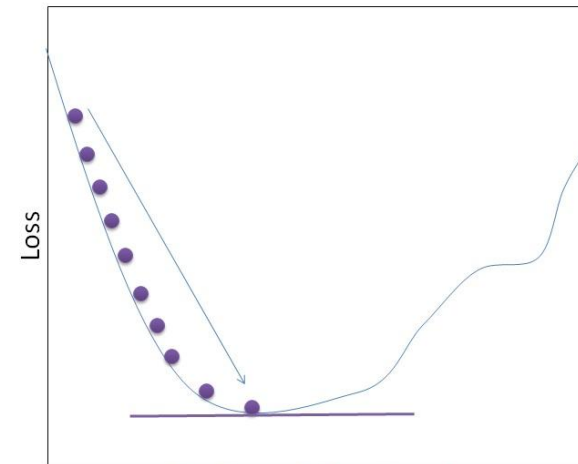
Learning Rate?



Too low learning rate



Too high learning rate



Good enough learning rate

How to deal with this?

Idea 1:

Try lots of different learning rates and see what works “just right”

How to deal with this?

Idea 1:

Try lots of different learning rates and see what works “just right”

Idea 2:

Do something smarter!

Design an adaptive learning rate that “adapts” to the landscape

Adaptive Learning Rates

- Learning rates are no longer fixed
- Can be made larger or smaller depending on:
 - how large gradient is
 - how fast learning is happening
 - size of particular weights
 - etc....

Gradient Descent Algorithms

Algorithm	TF Implementation	Torch Implementation	Reference
• SGD	 <code>tf.keras.optimizers.SGD</code>	 <code>torch.optim.SGD</code>	Kiefer & Wolfowitz, 1952.
• Adam	 <code>tf.keras.optimizers.Adam</code>	 <code>torch.optim.Adam</code>	Kingma et al., 2014.
• Adadelta	 <code>tf.keras.optimizers.Adadelta</code>	 <code>torch.optim.Adadelta</code>	Zeiler et al., 2012.
• Adagrad	 <code>tf.keras.optimizers.Adagrad</code>	 <code>torch.optim.Adagrad</code>	Duchi et al., 2011.
• RMSProp	 <code>tf.keras.optimizers.RMSProp</code>	 <code>torch.optim.RMSProp</code>	

Additional details: <http://ruder.io/optimizing-gradient-descent/>

Putting it all together



```
import tensorflow as tf

model = tf.keras.Sequential([...])

# pick your favorite optimizer
optimizer = tf.keras.optimizer.SGD()

while True: # loop forever

    # forward pass through the network
    prediction = model(x)

    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)

    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

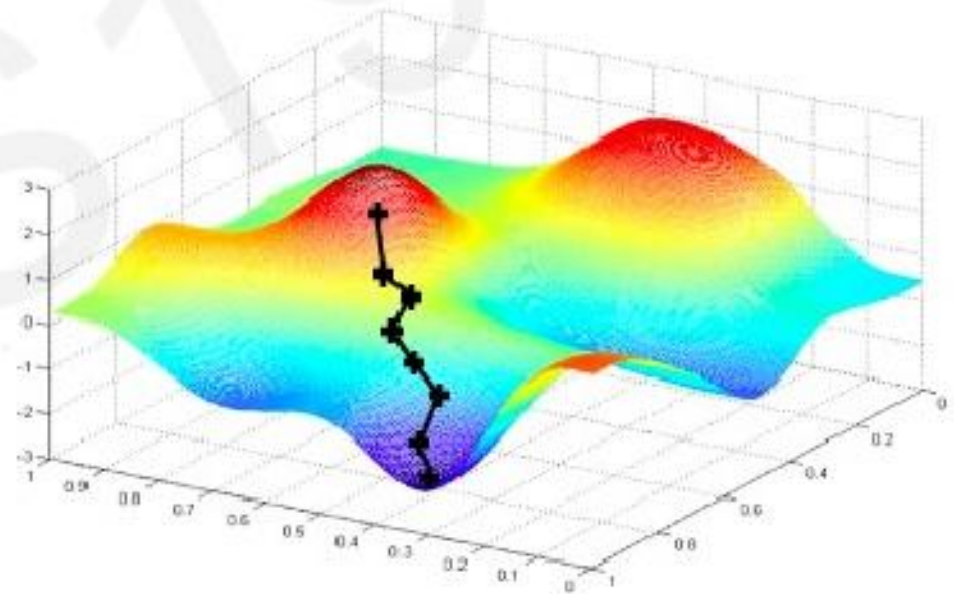


Can replace with
any TensorFlow
optimizer!

Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

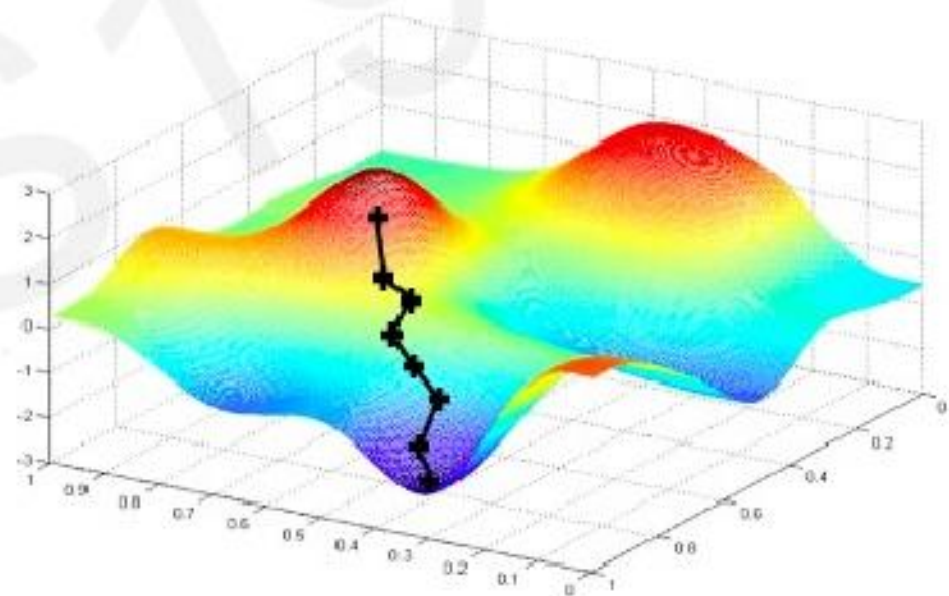


Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

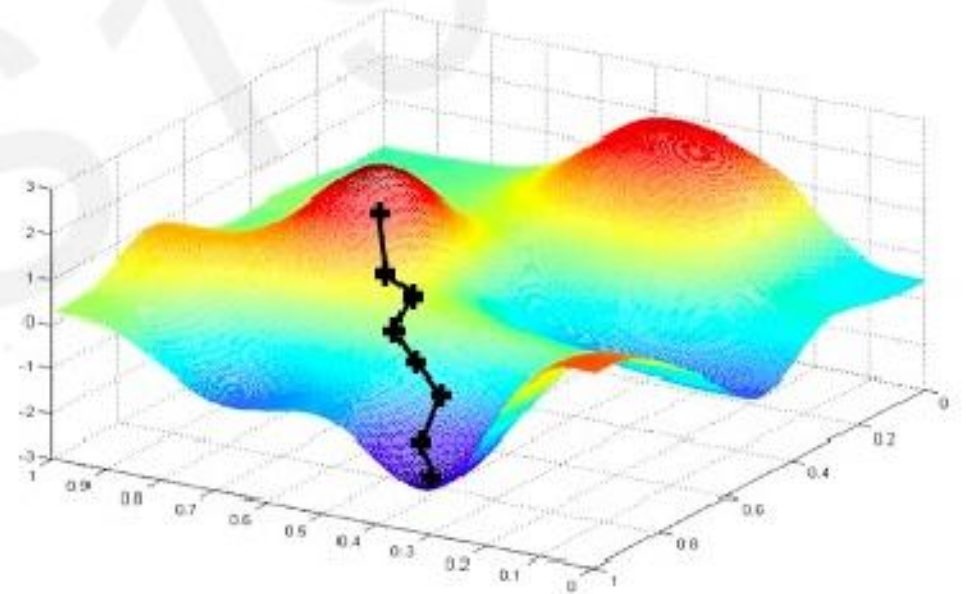
Can be very
computationally
intensive to compute!



Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights

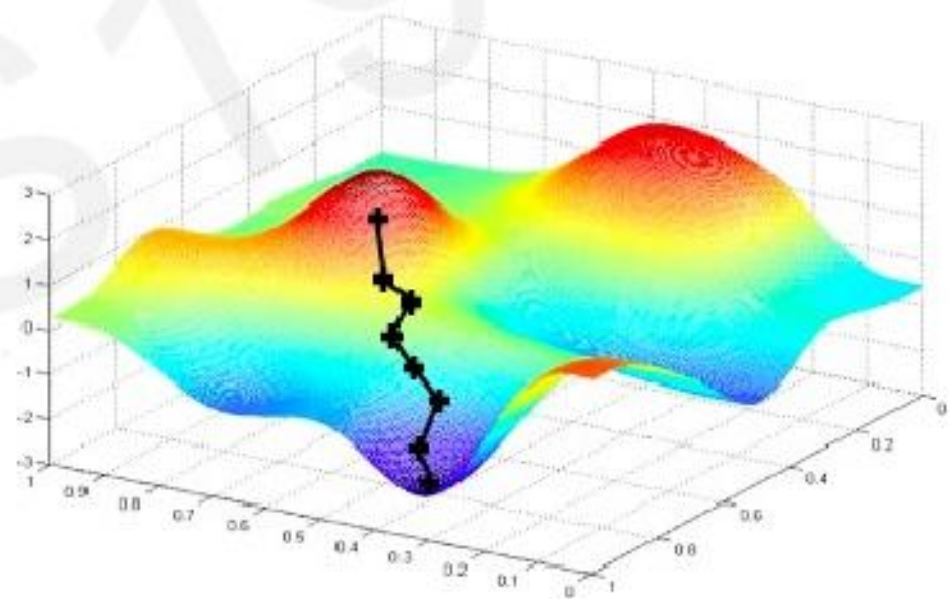


Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(W)}{\partial W}$
5. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights

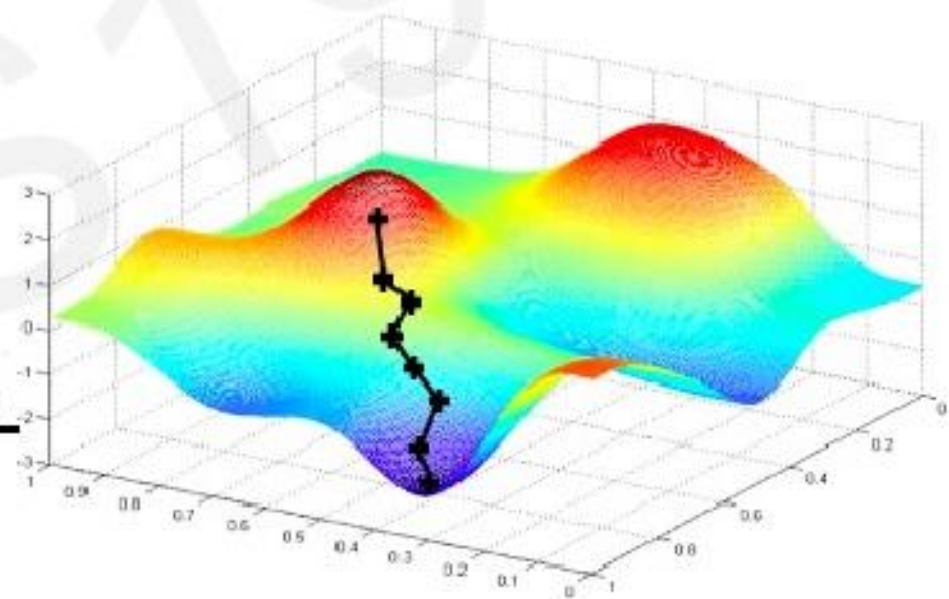
Easy to compute but
very noisy (stochastic)!



Mini-batches Gradient Descent

Algorithm

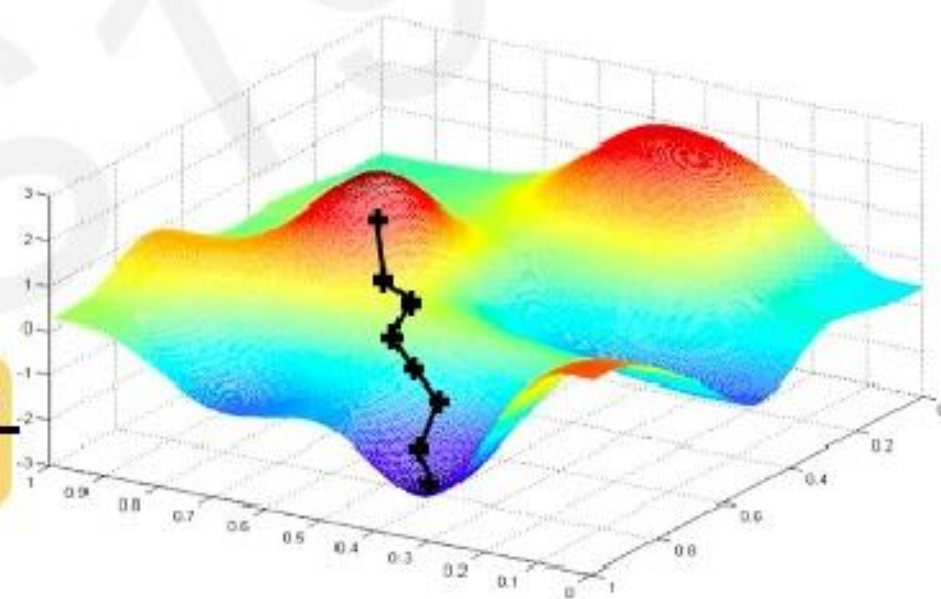
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Mini-batches Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$
5. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights



Fast to compute and a much better estimate of the true gradient!

Mini-batches while training

More accurate estimation of gradient

Smoother convergence
Allows for larger learning rates

Mini-batches while training

More accurate estimation of gradient

Smoother convergence
Allows for larger learning rates

Mini-batches lead to fast training!

Can parallelize computation + achieve significant speed increases on GPU's

Steps for A Perceptron

1. Perceptron Class (Variables & Functions)
2. Weighted sum (Summation Equation)
3. Activation Function (Non- Linearity)
4. Create the training method (How to adjust Weights & Biases)
5. Loading Data (Data & Label Separation, Training & Testing Data)
6. Training
7. Testing
8. Performance Evaluation (Accuracy)

Thank You

Any Questions?